Available online at www.sciencedirect.com
ScienceDirect
Comput. Methods Appl. Mech. Engrg. 393 (2022) 114764
Computer methods in applied mechanics and engineering
www.elsevier.com/locate/cma

A comparison of neural network architectures for data-driven reduced-order modeling

Anthony Gruber[a,*], Max Gunzburger[a], Lili Ju[b], Zhu Wang[b]

[a] *Department of Scientific Computing, Florida State University, 400 Dirac Science Library, Tallahassee, 32306, FL, USA*
[b] *Department of Mathematics, University of South Carolina, 1523 Greene Street, Columbia, 29208, SC, USA*

Received 5 October 2021; received in revised form 6 February 2022; accepted 14 February 2022
Available online 5 March 2022

Abstract

The popularity of deep convolutional autoencoders (CAEs) has engendered new and effective reduced-order models (ROMs) for the simulation of large-scale dynamical systems. Despite this, it is still unknown whether deep CAEs provide superior performance over established linear techniques or other network-based methods in all modeling scenarios. To elucidate this, the effect of autoencoder architecture on its associated ROM is studied through the comparison of deep CAEs against two alternatives: a simple fully connected autoencoder, and a novel graph convolutional autoencoder. Through benchmark experiments, it is shown that the superior autoencoder architecture for a given ROM application is highly dependent on the size of the latent space and the structure of the snapshot data, with the proposed architecture demonstrating benefits on data with irregular connectivity when the latent space is sufficiently large.

MSC: 65M22; 65M60

Keywords: Reduced-order modeling; Parametric PDEs; Graph convolution; Convolutional autoencoder; Nonlinear dimensionality reduction

1. Introduction

High-fidelity computational models are indispensable tools for the prediction and analysis of physical systems which are governed by parameterized partial differential equations (PDEs). As more and more industries are relying on simulated results to inform their daily operations, the significant amount of computational resources demanded by such models is becoming increasingly prohibitive. Indeed, actions which increase model fidelity such as refining the spatio-temporal resolution can also lead to an explosion of dimensionality, making use of the full-order model (FOM) infeasible in real-time or many-query scenarios. To remedy this, emphasis has been placed on reduced-order models (ROMs) which approximate the high-fidelity, full-order models at any desired configuration of parameters. Indeed, an appropriately built ROM can enable important applications such as uncertainty quantification or predictive control, which respectively require access to the FOM solution at thousands of points or near-instantaneous access to approximate solutions.

* Corresponding author.
E-mail addresses: agruber@fsu.edu (A. Gruber), mgunzburger@fsu.edu (M. Gunzburger), ju@math.sc.edu (L. Ju), wangzhu@math.sc.edu (Z. Wang).

https://doi.org/10.1016/j.cma.2022.114764
0045-7825/

The governing assumption inherent in the design of ROMs is that the FOM solution lies in a submanifold of the ambient space which is intrinsically low-dimensional. This is reasonable to posit when the FOM solution has been generated from the semi-discretization of an evolving quantity over a dense set of nodal points, as the values of this quantity in each dimension are strongly determined by the original (presumably lower-dimensional) dynamics. When this is the case, efficient approximation of the solution submanifold can lead to massive decreases in simulation time with relatively small approximation errors, since restriction to this space is lossless in principle. On the other hand, the problem of computing this submanifold is generally nontrivial, which has led to a wide variety of ROMs appearing in practice.

To enable this speedup in approximation of the FOM solution, ROMs are divided into offline and online stages. The offline stage is where the majority of expensive computation takes place, and may include operations such as collecting snapshots of the high-fidelity model, generating a reduced basis, or training a neural network. On the other hand, an effective ROM has an online stage that is designed to be as fast as possible, involving only a small amount of function evaluations or the solution of small systems of equations. Classically, most ROMs have relied on some notion of reduced basis to accomplish this task, which is designed to capture all of the important information about the high-fidelity system. For example, the well studied method of proper orthogonal decomposition (POD) described in Section 2 uses a number of solution snapshots to generate a reduced basis of size n whose span has minimal ℓ_2 reconstruction error. The advantage of this is that the high-fidelity problem can then be projected into the reduced space, so that only its low-dimensional representation must be manipulated at solving time. On the other hand, POD like many reduced-basis methods is a fundamentally linear technique, and therefore struggles in the presence of advection-dominated phenomena where the solution does not decay over time.

1.1. Related work

Desire to break the accuracy barrier of linear methods along with the rapid advancement of neural network technology has reignited interest in machine learning methods for model order reduction. In particular, it was first noticed in Milano and Koumoutsakos [1] that the reduced basis built with a linear multi-layer perceptron (MLP) model is equivalent to POD, and that nonlinear MLP offers improved prediction and reconstruction abilities at additional computational cost. Later, this idea was extended in works such as [2,3] which established that a fully connected autoencoder architecture can be trained to provide a nonlinear analogue of projection which often realizes significant performance gains over linear methods such as POD when the reduced dimension is small.

Following the mainstream success of large-scale parameter sharing network architectures such as convolutional neural networks, the focus of nonlinear ROMs began to shift in this direction. Since many high-fidelity PDE systems such as large-scale fluid simulations involve a huge number of degrees of freedom, the ability to train an effective network at a fraction of the memory cost for a fully connected network is very appealing and led to several important breakthroughs. The work of Lee and Carlberg in [4] demonstrated that deep convolutional autoencoders (CAEs) can overcome the Kolmogorov width barrier for advection-dominated systems which limits linear ROM methods, leading to a variety of similar ROMs based on this architecture seen in e.g. [5–8]. Moreover, works such as [6,7] have experimented with entirely data-driven ROMs based on deep CAEs, and seen success using either fully connected or recurrent long short term networks to simulate the reduced low-dimensional dynamics. At present, there are a large variety of nonlinear ROMs based on fully connected and convolutional autoencoder networks, and this remains an active area of research.

Despite the now widespread use of autoencoder networks for reduced-order modeling, there have not yet been studies which compare the performances of fully connected and convolutional networks in a ROM setting. In fact, many high-fidelity simulations take place on domains which are unstructured and irregular, so it is somewhat surprising that CAEs still perform well for this type of problem (see e.g. [7, Remark 5]). On the other hand, there is clearly a conceptual issue with applying the standard convolutional neural network (c.f. Section 2) to input data with variable connectivity; since the convolutional layers in this architecture accept only rectangular data, the input must be reshaped in a way which can change its neighborhood relationships, meaning the convolution operation may no longer be localized in space. This is a known issue that has been thoroughly considered in the literature on graph convolutional neural networks (see e.g. [9] and references therein), which have been successful in a wide range of classification tasks.

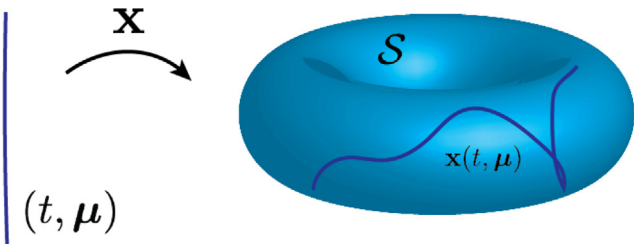

Fig. 1. An illustration of the solution submanifold $\mathcal{S} \subset \mathbb{R}^n$ parameterized by the quantities $(t, \boldsymbol{\mu})$.

1.2. Contributions

The goal of this work is to augment the ROM literature with a comparison between ROM-autoencoder architectures based on fully connected and convolutional neural networks. In particular, we compare the most popular deep convolutional autoencoder architecture from e.g. [4,7] to two alternatives: a basic fully connected autoencoder with batch normalization [10], and a novel autoencoder based on the GCNII graph convolution operation of [11]. Through benchmark experiments on regular and irregular data, we study the effectiveness of each architecture in compressing/decompressing high-fidelity PDE data, and evaluate its performance as part of a data-driven ROM. Results show that the superior architecture is highly dependent on the size of the reduced latent space as well as the modeling task involved. Moreover, some of the assumed advantages of the CAE (such as reduced memory consumption and easier training) are not satisfied in every case, and for some applications it is advantageous to use one of the other available architectures or a linear technique such as POD. It is our hope that the experiments provided here will enable modelers to make a more informed choice regarding which network architecture they employ for their autoencoder-based ROM.

2. Preliminaries

Consider a full-order model which is a dynamical system of parameterized ODEs,

$$\dot{\mathbf{x}}(t, \boldsymbol{\mu}) = \mathbf{f}\,(t, \mathbf{x}(t, \boldsymbol{\mu}), \boldsymbol{\mu})\,, \qquad \mathbf{x}(0, \boldsymbol{\mu}) = \mathbf{x}_0(\boldsymbol{\mu}). \tag{1}$$

Here $t \in [0, T]$ is time ($T > 0$), $\boldsymbol{\mu} \in D \subset \mathbb{R}^{n_\mu}$ is the vector of model parameters, $\mathbf{x} : [0, T] \times D \to \mathbb{R}^N$ is the parameterized state function with initial value $\mathbf{x}_0 \in \mathbb{R}^N$, and $\mathbf{f} \in [0, T] \times \mathbb{R}^N \times D \to \mathbb{R}^N$ is the state velocity. In practice, models (1) arise naturally from the semi-discretization of a parameterized system of PDE, where the discretization of the domain creates an N-length vector (or array) $\mathbf{x}$ representing a quantity defined at the nodal points. Clearly, a semi-discretization of size N has the potential to create a large amount of redundant dimensionality, as a unique dimension is created for every nodal point while the solution remains characterized by the same number of parameters.

To effectively handle the high-dimensional ODE (1), the overarching goal of reduced-order models is to predict approximate solutions without solving the governing equations in the high-dimensional space $\mathbb{R}^N$. In principle, this is possible as long as the assignment $(t, \boldsymbol{\mu}) \mapsto \mathbf{x}(t, \boldsymbol{\mu})$ is unique for each $(t, \boldsymbol{\mu})$, meaning that the space of solutions spans a low-dimensional manifold in the ambient $\mathbb{R}^N$ (c.f. Fig. 1). Therefore, the ROM should provide fast access to the solution manifold

$$\mathcal{S} = \{\mathbf{x}(t, \boldsymbol{\mu}) \mid t \in [0, T],\ \boldsymbol{\mu} \in D\} \subset \mathbb{R}^N,$$

which contains integral curves of the high-dimensional FOM. Note that the uniqueness assumption on $\mathbf{x}$ implies that the coordinates $(t, \boldsymbol{\mu})$ form a global chart for $\mathcal{S}$ and hence $\mathcal{S}$ is intrinsically $(n_\mu + 1)$-dimensional (at most), which is frequently much less than the ambient dimension N.

Traditionally, many ROMs are generated through some form of Proper Orthogonal Decomposition (POD). To apply POD, some number of solutions to (1) are first collected and stored in a snapshot matrix $\mathbf{S} = (s^i_j) \in \mathbb{R}^{N \times N_s}$ where N_s is the number of training samples, $s^i_j = x^i(t_j, \boldsymbol{\mu}_j)$, x^i represents the ith component of the solution $\mathbf{x}$, and nonuniqueness is allowed in the parameters $\boldsymbol{\mu}_j$. A singular value decomposition of the snapshot matrix $\mathbf{S} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$

is then performed, and the first n columns $\mathbf{U}_n$ of $\mathbf{U} \in \mathbb{R}^{N\times N}$ (corresponding to the largest n eigenvalues of Σ) are chosen to serve as a reduced basis for the solution space. Geometrically, this procedure amounts to choosing as basis an n-plane of POD modes in $\mathbb{R}^N$ (the columns of $\mathbf{U}_n$) which minimizes the ℓ_2 projection error $\left\|\mathbf{x} - \mathbf{U}_n\mathbf{U}_n^\top\mathbf{x}\right\|$ over the snapshot data. A ROM can then be generated from this by projecting (1) to this n-plane. More precisely, writing the approximate solution $\tilde{\mathbf{x}} = \mathbf{U}_n\hat{\mathbf{x}}$ for some $\hat{\mathbf{x}} \in \mathbb{R}^n$ and using the FOM along with the fact that $\mathbf{U}_n^\top\mathbf{U}_n = \mathbf{I}_n$ leads to the system

$$\dot{\hat{\mathbf{x}}}(t,\boldsymbol{\mu}) = \mathbf{U}_n^\top\mathbf{f}\left(t, \mathbf{U}_n\hat{\mathbf{x}}, \boldsymbol{\mu}\right), \qquad \hat{\mathbf{x}}(0,\boldsymbol{\mu}) = \hat{\mathbf{x}}_0(\boldsymbol{\mu}) = \mathbf{U}_n^\top\mathbf{x}_0(\boldsymbol{\mu}),$$

which requires the solution of only $n \ll N$ equations.

POD based techniques are known to be highly effective in many cases, especially when the problem dynamics are diffusion-dominated. On the other hand, advection-dominated problems exhibit a slowly decaying Kolmogorov n-width

$$d_n(\mathcal{S}) = \inf_{M_n}\sup_{\mathbf{x}\in\mathcal{S}}\inf_{\hat{\mathbf{x}}\in M_n}\|\mathbf{x} - \hat{\mathbf{x}}\|,$$

meaning a large number of POD modes is required for accurate approximation of their solutions. This reveals a fundamental issue with linear approximation spaces which motivated recent study into neural network based ROMs. By now, it is well known that the barrier posed by d_n can be surmounted using deep convolutional autoencoders, which are now reviewed.

2.1. Convolutional autoencoder networks

Recall that a fully connected neural network (FCNN) is a function $\mathbf{f} = \mathbf{f}_n \circ \mathbf{f}_{n-1} \circ \ldots \circ \mathbf{f}_1$ of the input vector $\mathbf{x} = \mathbf{x}_0 \in \mathbb{R}^{N_0}$ such that at layer ℓ,

$$\mathbf{x}_\ell := \mathbf{f}_\ell(\mathbf{x}_{\ell-1}) = \boldsymbol{\sigma}_\ell\left(\mathbf{W}_\ell\mathbf{x}_{\ell-1} + \mathbf{b}_\ell\right), \tag{2}$$

where $\boldsymbol{\sigma}_\ell = \mathrm{Diag}(\sigma_\ell)$ is a nonlinear activation function acting row-wise on its input, $\mathbf{W}_\ell \in \mathbb{R}^{N_\ell\times N_{\ell-1}}$ is the weight matrix of the layer and $\mathbf{b} \in \mathbb{R}^{N_\ell}$ is its bias. FCNNs are extremely expressive due to their large amount of parameters, but for the same reason they are typically difficult to train and incur a high memory cost. On the other hand, a convolutional neural network (CNN) can be considered a special case of FCNN where weights are shared and determined by a translation-invariant kernel function. More precisely, a convolutional neural network is a function of a multidimensional input $\mathbf{x} = \mathbf{x}_0$ such that at layer ℓ,

$$\mathbf{x}_{\ell,i} = \boldsymbol{\sigma}_\ell\left(\sum_{j=1}^{C_{in}}\mathbf{x}_{(\ell-1),j} * \mathbf{W}_{\ell,i}^j + \mathbf{b}_{\ell,i}\right),$$

where $1 \le i \le C_{out}$ denotes the spatially-independent "channel dimension" of $\mathbf{x}$ which may differ at each layer (i.e. $C_{in} \ne C_{out}$), $\mathbf{W}_{\ell,i}^j$ is the convolution kernel at layer ℓ, and $\mathbf{b}_{\ell,i}$ is a bias object which is constant for each fixed ℓ, i. Note that the kernel $\mathbf{W}$ has tensor dimension equal to that of the input plus one, since it depends on the shape of $\mathbf{x}_{\ell-1}$ as well as the number of channels at layers $\ell, \ell-1$. Here $*$ plays the role of the discrete convolution operator, defined for two-dimensional, one-channel arrays $\mathbf{x}, \mathbf{W}$ as

$$(\mathbf{x} * \mathbf{W})_\beta^\alpha = \sum_{\gamma,\delta} x_{(s\beta+\delta)}^{(s\alpha+\gamma)}\, w_{(M-1-\delta)}^{(L-1-\gamma)},$$

where $s \in \mathbb{N}$ is the stride length, $\mathbf{W} = (w_\delta^\gamma) \in \mathbb{R}^L \times \mathbb{R}^M$, and all Greek indices are compactly supported and begin at zero. It follows from this definition of the discrete convolution that the ranges of α, β in the output depend on the constants s, M, L as well as the shape of the input. To determine this more precisely, consider projecting the tensor $\mathbf{x} * \mathbf{W}$ along some coordinate direction and computing the length of this projection in terms of the lengths of the corresponding projections of $\mathbf{x}, \mathbf{W}$. If the projection of $\mathbf{x}$ along this direction has length I and the projection of $\mathbf{W}$ has length $F \le I$, then it is straightforward to show [12, Relationship 5] that the output projection has length $1 + \left\lfloor\frac{I-F}{s}\right\rfloor$. Therefore, convolution will naturally decrease the dimensionality of the input data $\mathbf{x}$ when $F \ge 2$, creating a downsampling effect which is highly useful for techniques such as image compression. This

downsampling can be further controlled by padding the input $\mathbf{x}$ with an array of zeros, in which case the size of the output changes based on other calculable rules (see e.g. [12]).

Since the discrete convolution (2) decreases the dimensionality of the input data in a controlled way, it is reasonable to make use of it when constructing convolutional autoencoder (CAE) networks. In particular, the main idea behind CAE networks is to stack convolutional layers in such a way that the input $\mathbf{x}$ is successively downsampled (or encoded) to a low-dimensional representation, and then upsampled (or decoded) back to its original form. Combined with an appropriate notion of reconstruction error to be minimized, this encourages the neural network to learn an approximation to the identity mapping $\mathbf{x} \approx \tilde{\mathbf{x}} = \mathbf{g} \circ \mathbf{h}(\mathbf{x})$ which is useful for tasks such as compressed sensing and PDE approximation provided the decoder $\mathbf{g}$ can be decoupled from the encoder $\mathbf{h}$. In the context of ROM, the notion of data downsampling serves as a nonlinear form of projection analogous to the truncated eigenvalue decomposition in POD. In principle, as more characteristic features are extracted by the convolutional layers in the network, less dimensionality is required to encode the original solution $\mathbf{x}$. Therefore, repeated convolution enables the most basic features of $\mathbf{x}$ to be represented with a latent space of small dimension, which can be exploited by the network for accurate reconstruction of the solution manifold. Moreover, the assumption of uniqueness in the solution mapping $(t, \boldsymbol{\mu}) \mapsto \mathbf{x}(t, \boldsymbol{\mu})$ guarantees that this idea is loss-less whenever the latent dimension is at least as large as $n_\mu + 1$.

On the other hand, if convolution is to be useful for ROM, it is necessary to have a way to upsample (or decode) the latent representation for $\mathbf{x}$. Although the convolution operation is clearly not invertible, it can be transposed in a way which is useful for this purpose. In particular, note that discrete convolution can be considered a matrix operation by lexicographically "unrolling" the inputs $\mathbf{x}$ into a vector and similarly unrolling the kernel $\mathbf{W}$ into a sparse matrix which acts on that vector. The operator $\mathbf{W}$ then has a well defined transpose, which effectively broadcasts the (unrolled) low-dimensional input to a high-dimensional output which can be rolled back into its original shape. In practice, this operation is accomplished by simply exchanging the forward and backward passes of the relevant convolutional layer, as it can be shown (see [12, Section 4]) that this is operationally equivalent to computing the transposed convolution. Doing this several times in succession produces the required upsampling effect, and traditionally makes up the decoder part of the CAE structure.

2.2. Autoencoder-based ROMs

As mentioned before, using a well trained CAE $\mathbf{g} \circ \mathbf{h}$ in place of the linear reconstruction $\mathbf{U}_n \mathbf{U}_n^\top$ when designing a ROM can provide a great benefit to accuracy. In particular, examples from [4,7] show that deep CAEs can approach the theoretical minimum error with only a few latent variables, enabling improved accuracy over POD-ROMs when the number of modes is small. Moreover, autoencoder-based ROMs do not require any more information than standard POD-ROMs. Indeed, developing a ROM using the decoder mapping $\mathbf{g}$ means seeking $\tilde{\mathbf{x}} \approx \mathbf{x}$ which satisfies

$$\tilde{\mathbf{x}}(t, \boldsymbol{\mu}) = \mathbf{g}\left(\hat{\mathbf{x}}(t, \boldsymbol{\mu})\right),$$

where $\hat{\mathbf{x}} : \mathbb{R}^{n_\mu + 1} \to \mathbb{R}^n$, $n \ll N$ is a latent representation of the state $\mathbf{x}$, and $\mathbf{g} : \mathbb{R}^n \to \mathbb{R}^N$ is a nonlinear mapping from the latent space to the ambient $\mathbb{R}^N$. In particular, if $\mathbf{g}$ can be constructed such that its image is the solution submanifold $\mathcal{S}$, then the comparatively small latent representation $\hat{\mathbf{x}}$ is sufficient for complete characterization of the state $\mathbf{x}$. As $\mathbf{g}$ is typically trained using only the snapshots in $\mathbb{S}$, when $n \geq n_\mu + 1$ this provides the potential for an accurate nonlinear mapping using the same information necessary for POD.

Using the approximation $\tilde{\mathbf{x}}$ in the FOM (1), it follows that $\dot{\tilde{\mathbf{x}}} = \mathbf{g}'(\hat{\mathbf{x}})\dot{\hat{\mathbf{x}}}$ where $\mathbf{g}'(\hat{\mathbf{x}})$ denotes the derivative operator (i.e. Jacobian matrix) of $\mathbf{g}$. Therefore, a reduced order model involving only $\hat{\mathbf{x}}, \mathbf{g}$ can be generated by assuming that the approximate state $\tilde{\mathbf{x}}$ obeys the same dynamics as the original state $\mathbf{x}$. In particular, if $\dot{\tilde{\mathbf{x}}} = \mathbf{f}$ and $\mathbf{g}'$ has full rank at every $\hat{\mathbf{x}}$, it follows quickly that

$$\dot{\hat{\mathbf{x}}}(t, \boldsymbol{\mu}) = \mathbf{g}'(\hat{\mathbf{x}})^+ \mathbf{f}\left(t, \mathbf{g}(\hat{\mathbf{x}}), \boldsymbol{\mu}\right), \qquad \tilde{\mathbf{x}}(0, \boldsymbol{\mu}) = \tilde{\mathbf{x}}_0(\boldsymbol{\mu}) = \mathbf{g}\left(\hat{\mathbf{x}}(0, \boldsymbol{\mu})\right), \tag{3}$$

where $(\mathbf{g}')^+ = (\mathbf{g}'^\top \mathbf{g}')^{-1} \mathbf{g}'^\top$ denotes the Moore–Penrose pseudoinverse of $\mathbf{g}'$. This is a well posed low-dimensional dynamical system that is feasible to solve when $\mathbf{g}$ and its derivatives are known. Note that the ROM (3) takes place on the tangent bundle to the solution manifold $\mathcal{S}$, so that $\hat{\mathbf{x}}$ is a low-dimensional integral curve in this space. Moreover, it is clear that choosing the mapping $\mathbf{g} = \mathbf{U}_n$ immediately recovers the standard POD-ROM from before since $\mathbf{U}_n$ is a linear mapping.

Solving the ROM (3) is often accomplished using Newton or quasi-Newton methods based on minimizing the residual (e.g. [4,13]), though recent work in [6,7] has demonstrated that a purely data-driven approach can also be effective for end-to-end reduced-order modeling of PDEs without appealing to conventional numerical solvers. More precisely, instead of relying on a low-dimensional system of differential equations to compute the latent representation $(t, \boldsymbol{\mu}) \mapsto \hat{\mathbf{x}}(t, \boldsymbol{\mu})$, it is reasonable to consider computing this quantity with another neural network which is fully connected and relatively simple. In this case, the autoencoder $\mathbf{g} \circ \mathbf{h}$ can be trained alongside the mapping $\hat{\mathbf{x}}$ through the joint loss

$$L(\mathbf{x}, t, \boldsymbol{\mu}) = \|\mathbf{x} - \mathbf{g} \circ \mathbf{h}(\mathbf{x})\|^2 + \|\hat{\mathbf{x}}(t, \boldsymbol{\mu}) - \mathbf{h}(\mathbf{x})\|^2. \tag{4}$$

The first term in L is simply the reconstruction error coming from the autoencoder, while the second term encourages the latent representation $\hat{\mathbf{x}}$ to agree with the encoded state $\mathbf{h}(\mathbf{x})$. This approach has the advantage of requiring only function evaluations in its online stage, at the potential cost of model generalizability. On the other hand, experiments in [7] as well as Section 4 of this work show that data-driven ROMs still produce competitive results in practice while in some cases maintaining a lower online cost.

Remark 2.1. Alternatively to using the joint loss (4), its composite terms can be used separately to train the mapping $\hat{\mathbf{x}}$ after the mappings $\mathbf{g}, \mathbf{h}$ in a two-stage process. However, in practice we observe similar to [7] that network ROM performance is not significantly affected by this choice. This could be due to the observed accuracy bottleneck coming from $\hat{\mathbf{x}}$ (c.f. Section 4).

It is important to keep in mind that the existence of the decoder mapping $\mathbf{g}$ is independent of the numerical technique used to generate it, and therefore the use of the convolutional propagation rule (2) is a design decision rather than a necessity. In fact, at present there are an enormous number of different neural network architectures available for accomplishing both predictive and generative tasks, each with their own unique properties. Therefore, it is reasonable to question whether or not deep CAEs based on (2) provide the best choice of network architecture for ROM applications. The goal of the sequel is to introduce an alternative to the conventional CAE, along with some experiments which compare the performance of this CAE-ROM to other autoencoder-based ROMs involving fully connected and graph convolutional neural networks.

3. A graph convolutional autoencoder for ROM

The convolutional propagation rule (2) comes with inherent advantages and disadvantages that become readily apparent in a ROM setting. One very positive quality of this rule is that if the convolutional filter is of size $K \times K$, then its associated discrete convolution is precisely K-localized in space. This locality ensures that only the features of nearby nodes are convolved together, increasing the interpretability and expressiveness of the neural network. On the other hand, it is clear that (2) does not translate immediately to irregular domains, where each node has a variable number of neighbors in its K-hop neighborhood. Because of this, bothersome tricks are needed to apply the CAE based on (2) to PDE data defined on irregular or unstructured domains. In particular, one popular approach (c.f. [7]) is to simply pad and reshape the solution. More precisely, each feature of $\mathbf{x}$ can be flattened into a vector, zero-padded until it has length 4^m for some m, and then reshaped to a square of size $2^m \times 2^m$. This square is then fed into the CAE, generating an output of the same shape which can be flattened and truncated to produce the required feature of $\tilde{\mathbf{x}}$. While this procedure works surprisingly well in many cases, it has the potential to create a very high overhead cost due to artificial padding of the solution. Moreover, it fails to preserve the beneficial locality property of convolution, since nodes which were originally far apart may end up close together after reshaping. Remedying these issues requires a new autoencoder architecture based on graph convolutional neural networks, which will now be discussed.

3.1. Graph convolutional neural networks

While the discrete convolution in (2) is in some sense a direct translation of the continuous version, it is only meaningful for regular, structured data. However, PDE simulations based on high-fidelity numerical techniques such as finite element methods frequently involve domains which are irregular and unstructured. In this case, developing an autoencoder-based ROM requires a notion of convolution which is appropriate for data with arbitrary connectivity.

On the other hand, it is not immediately obvious how to define a useful notion of convolution when the domain in question is unstructured. Because discrete procedures typically obey a "no free lunch" scenario with respect to their continuous counterparts, it is an active area of research to determine which properties of the continuous convolution are most beneficial when preserved in the discrete setting. Since unstructured domains can be effectively modeled as mathematical graphs, this has motivated a large amount of activity in the area of graph convolutional neural networks (GCNNs). The next goal is to recall the basics of graph convolution and propose an autoencoder architecture which makes use of this technology.

To that end, consider an undirected graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ with (weighted or unweighted) adjacency matrix $\mathbf{A} \in \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|}$. Recall that $\mathbf{A} = (a_{ij})$ is symmetric with entries defined by

$$a_{ij} = \begin{cases} w_{ij} & \exists\, e_{ij} \in \mathcal{E} \text{ connecting } v_i \text{ and } v_j \\ 0 & \text{otherwise,} \end{cases}$$

where $w_{ij} > 0$ are the weight values associated to each edge. Given this information, there are a wide variety of approaches to defining a meaningful convolution between two signals $f, g : \mathcal{V} \to \mathbb{R}^n$ defined at the nodes of $\mathcal{G}$ (see e.g. [9]). One highly successful approach is to make use of the fact that convolution is just ordinary multiplication in Fourier space. In particular, recall the combinatorial Laplacian $\mathbf{L} : \mathbb{R}^{|\mathcal{V}|} \to \mathbb{R}^{|\mathcal{V}|}$ which can be computed as $\mathbf{L} = \mathbf{D} - \mathbf{A}$ where $\mathbf{D} = (d_{ij})$ is the diagonal degree matrix with entries $d_{ii} = \sum_j a_{ij}$. It is straightforward to check that $\mathbf{L}$ is symmetric and positive semi-definite (see e.g. [14]), therefore $\mathbf{L} = \mathbf{U}\Lambda\mathbf{U}^\top$ has a complete set of real eigenvalues λ_i with associated orthonormal eigenvectors $\mathbf{u}_i$ known as the Fourier modes of $\mathcal{G}$. Since any signal $f : \mathcal{V} \to \mathbb{R}^n$ can be considered an n-length array of $|\mathcal{V}|$-vectors denoted $\mathbf{f} \in \mathbb{R}^{n\times|\mathcal{V}|}$, this yields the discrete Fourier transform $\hat{\mathbf{f}}_j = \mathbf{U}^\top\mathbf{f}_j$ where $\mathbf{f}_j \in \mathbb{R}^{|\mathcal{V}|}$ is the jth vectorized component function of f, and each entry $\hat{f}_j(\lambda_i) = \langle \mathbf{u}_i, \mathbf{f}_j \rangle$. The advantage of this notion is a straightforward extension of convolution to graphical signals $f, g : \mathcal{V} \to \mathbb{R}^n$ through multiplication in frequency space. In particular, it follows that for $1 \le i, j \le n$

$$\mathbf{f}_i * \mathbf{g}_j = \mathbf{U}\left(\mathbf{U}^\top\mathbf{f}_i \odot \mathbf{U}^\top\mathbf{g}_j\right),$$

where $\odot$ denotes the element-wise Hadamard product and $\mathbf{f}_i = \mathbf{U}\hat{\mathbf{f}}_i$ is the inverse Fourier transform on $\mathcal{G}$. Note the usual convention that matrix products occur before the Hadamard product.

This expression provides a mathematically precise (though indirect) definition of spatial graph convolution through the graph Fourier transform, but carries the significant drawback of requiring matrix multiplication with the Fourier basis $\mathbf{U}$ which is generically non-sparse. To remedy this, Defferrard et al. [15] proposed considering matrix-valued filters which can be recursively computed from the Laplacian $\mathbf{L}$. In particular, consider a convolution kernel $g_\theta(\mathbf{L}) = \mathbf{U}g_\theta(\Lambda)\mathbf{U}^T$ where $g_\theta(\Lambda) = \sum_{k=0}^K \theta_k\Lambda^k$ is an order K polynomial in the eigenvalues of the *normalized* Laplacian $\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$ and θ is a vector of learnable coefficient parameters. Such kernels are easily computed if the eigenvalues Λ are known, but do not obviously eliminate the need for multiplication by the Fourier basis $\mathbf{U}$. On the other hand, if $g_\theta(\mathbf{L})$ can be computed recursively from $\mathbf{L}$, then for any signal $\mathbf{x} \in \mathbb{R}^{|\mathcal{V}|}$ the convolution $g_\theta * \mathbf{x} = g_\theta(\mathbf{L})\mathbf{x}$ can be computed in $\mathcal{O}(K|\mathcal{E}|) \ll \mathcal{O}(|\mathcal{V}|^2)$ operations, making the spectral approach more feasible for practical neural network applications. As an added benefit of the work in [15], note that order-K polynomial kernels are also precisely K-localized in space. In particular, the jth component of the kernel g_θ localized at vertex i is given by

$$\left(g_\theta(\mathbf{L})\boldsymbol{\delta}_i\right)_j = \left(g_\theta(\mathbf{L})\right)_{ij} = \sum_{k=0}^K \theta_k(\mathbf{L}^k)_{ij},$$

where $\boldsymbol{\delta}_i$ is the ith nodal basis vector and $(\mathbf{L}^k)_{ij} = 0$ whenever v_j lies outside the K-hop neighborhood of v_i (c.f. [16, Lemma 5.2]). This shows that the frequency-based convolution $g_\theta(\mathbf{L})\mathbf{x}$ maintains the desirable property of spatial locality when the filter is a polynomial in $\mathbf{L}$.

Further simplification to this idea was introduced by Kipf and Welling [17] who chose $K = 1, \theta_0 = 2\theta$, and $\theta_1 = -\theta$ with θ a learnable parameter to obtain the first-order expression $g_\theta(\mathbf{L})\mathbf{x} = \theta\left(\mathbf{I} + \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}\right)\mathbf{x}$. By renormalizing the initial graph to include self-loops (i.e. adding identity to the adjacency and degree matrices), they proposed the graph convolutional network (GCN) which obeys the propagation rule

$$\mathbf{x}_{\ell+1} = \boldsymbol{\sigma}\left(\tilde{\mathbf{P}}\mathbf{x}_\ell\mathbf{W}_\ell\right), \tag{5}$$

where $\mathbf{x}_\ell \in \mathbb{R}^{|\mathcal{V}|\times n_\ell}$ is a signal on the graph with n_ℓ channels, $\mathbf{W}_\ell \in \mathbb{R}^{n_\ell \times n_{\ell+1}}$ is a (potentially nonsquare) weight matrix containing the learnable parameters and

$$\tilde{\mathbf{P}} = \tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2} = (\mathbf{D}+\mathbf{I})^{-1/2}(\mathbf{A}+\mathbf{I})(\mathbf{D}+\mathbf{I})^{-1/2}.$$

Networks of this form are simple and fast to compute, but were later shown [18] to be only as expressive as a fixed polynomial filter $g(\mathbf{\Lambda}) = (\mathbf{I}-\mathbf{\Lambda})^K$ on the spectral domain of the self-looped graph with order equal to the depth K of the network. While the propagation rule (5) displayed good performance on small-scale graphs, it is now known that simple GCNs are notorious for overfitting and oversmoothing the data, as their atomic operation is related to heat diffusion on the domain. This severely limits their expressive power, as GCNNs based on (5) are depth-limited in practice and cannot effectively reconstruct complex signals when used in a ROM setting.

On the other hand, many other graph convolutional operations are available at present, some of which were developed specifically to remedy this aspect of the original GCN. One such operation was introduced in [11] and directly extends approach of [17]. In particular, the authors of [11] define the "GCNII" or GCN2 network, whose propagation rule is

$$\mathbf{x}_{\ell+1} = \boldsymbol{\sigma}\left[\left((1-\alpha_\ell)\tilde{\mathbf{P}}\mathbf{x}_\ell + \alpha_\ell \mathbf{x}_0\right)\left((1-\beta_\ell)\mathbf{I} + \beta_\ell \mathbf{W}_\ell\right)\right], \tag{6}$$

where α_ℓ, β_ℓ are hyperparameters and $\boldsymbol{\sigma}$ is the component-wise ReLU function $\sigma(x) = \max\{x, 0\}$. The terms in this operation are motivated by the ideas of residual connection and identity mapping popularized in the well known ResNet architecture (c.f. [19]), and the authors prove that K-layer networks of this form can express an arbitrary Kth-order polynomial in the self-looped graph Laplacian. While the hyperparameters α_ℓ are typically constant in practice, it is noted in [11] that improved performance is attained when β_ℓ decreases with increasing network depth. Therefore, the experiments in Section 4 use the recommended values of $\alpha_\ell = 0.2$ and $\beta_\ell = \log(1+\frac{\theta}{\ell})$ where $\theta = 1.5$ is a constant hyperparameter.

With this, it is now possible to use the propagation rule (6) as the basis for a convolutional autoencoder. Although spectral graph convolution does not produce any natural downsampling analogous to the CNN case, it is still reasonable to expect that a succession of convolutional layers will learn to extract important features of the data it accepts as input. Therefore, it should be possible to recover the decoder mapping $\mathbf{g}$ necessary for ROM from this approach as well. Moreover, in contrast to the discrete convolution (2) the GCN2 operation is invariant to label permutation and remains K-localized on unstructured data. Therefore, it is reasonable to suspect that a CAE-ROM based on (6) will exhibit superior performance when the PDE to be simulated takes place on an unstructured domain. The remainder of the work will investigate whether or not this is the case, beginning with a description of the GCNN autoencoder that will be used.

Remark 3.1. Note that there are a wide variety of graph convolutional operations which can be used as the basis for a convolutional autoencoder on unstructured data. During the course of this work, many such operations were investigated including the ones proposed in [20–24]. Through experimentation, it was determined that the graph convolution proposed in [11] is the most effective for the present purpose.

3.2. Graph convolutional autoencoder architecture

Recall that the standard CAE is structured so that each layer downsamples the input in space at the same rate that it extrudes the input in the channel dimension. In particular, it is common to use 5×5 convolution kernels with a stride of two and "same" zero padding (c.f. [12, Section 4.4]), so that the spatial dimension of $\mathbf{x}$ decreases by a factor of four at each layer. While this occurs, the channel dimension is simultaneously increased by a factor of four, so that the total dimensionality of $\mathbf{x}$ remains constant until the last layer(s) of the encoder. The final part of the encoder is often fully connected, taking the simplified output of the convolutional network and mapping it directly to the encoded representation $\mathbf{h}(\mathbf{x})$ in some small latent space $\mathbb{R}^n$. To recover $\tilde{\mathbf{x}} \approx \mathbf{x}$ from its encoding, this entire block of layers is simply "transposed" to form the decoder mapping $\mathbf{g}$, so that the autoencoder comprises the mapping $\mathbf{g} \circ \mathbf{h}$ as desired. The learnable parameters of the mappings $\mathbf{g}, \mathbf{h}$ are occasionally shared (resulting in a "tied autoencoder"), though most frequently they are not.

To study whether autoencoder-based ROMs benefit from the core ideas behind GCNNs, we propose the ROM architecture displayed in Fig. 2. Similar to [7], this ROM is entirely data-driven and is comprised of two neural

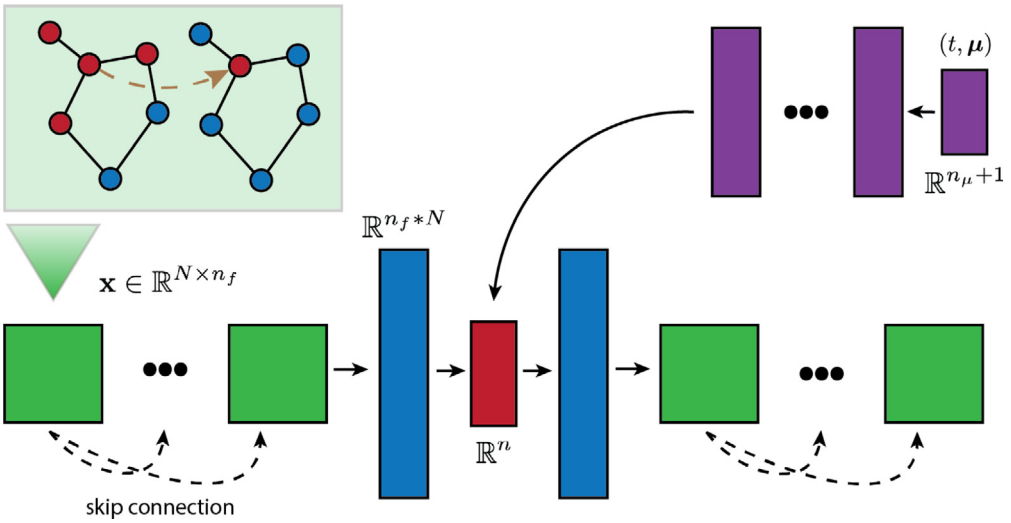

Fig. 2. Illustration of the ROM architecture based on GCN2 graph convolution.

networks: a specially designed CAE based on the operation (6) whose decoder is the required mapping $\mathbf{g}$, and a small-scale FCNN which simulates the low-dimensional mapping $(t, \boldsymbol{\mu}) \mapsto \hat{\mathbf{x}}(t, \boldsymbol{\mu})$. In this way, the component networks are trained directly from snapshot data during the offline stage, so that the online stage requires only function evaluations. Recall that the input $\mathbf{x}$ is not downsampled by the convolutional procedure (6), so the feature dimension is similarly held constant. Moreover, use of the GCN2 operation ensures that each convolutional layer is skip-connected to the first, i.e. each convolutional layer retains a fraction of the pre-convolutional input. Therefore, the proposed autoencoder is defended against the troublesome oversmoothing known to occur in deep GCNs. To evaluate the performance of our GCNN-ROM, it is compared directly to two analogous autoencoder-based data-driven ROMs which use the same prediction network $\hat{\mathbf{x}}$ and loss function (4) but different autoencoder architectures. The first is the standard deep CAE based on (2) and designed according to the common conventions of CAEs on regular data, and the second is a simple feed-forward autoencoder which uses batch-normalized fully connected layers throughout. The results of a comparison between these ROMs are given in the next Section.

4. Numerical examples

This section details a performance comparison between ROMs using autoencoder architectures based on fully connected (FCNN), convolutional (CNN), and graph convolutional (GCNN) networks. Through examples of PDEs on structured and unstructured domains, ROM performance is evaluated on two distinct tasks. The first task measures how well each architecture can approximate PDE solutions at new parameter values (referred to as the *prediction problem*), and the second measures how well they can compress and decompress PDE data (referred to as the *compression problem*).

4.1. Data preprocessing and evaluation

Begin with a snapshot array $\mathbf{S} \in \mathbb{R}^{N_s\times N\times n_f}$ each row of which is a solution $\mathbf{x}(t, \boldsymbol{\mu})$ at a particular parameter configuration $(t, \boldsymbol{\mu})$ found in the (consistently ordered) parameter matrix $\mathbf{M} \in \mathbb{R}^{N_s\times(n_\mu+1)}$. We first choose integers N_t, N_v such that $N_s = N_t + N_v$ and separate $\mathbf{M} = (\mathbf{M}_t, \mathbf{M}_v)$ and $\mathbf{S} = (\mathbf{S}_t, \mathbf{S}_v)$ row-wise into training and validation sets, taking care that no value of $\boldsymbol{\mu}$ appears in both $\mathbf{M}_t, \mathbf{M}_v$. After separation, the training data $\mathbf{M}_t, \mathbf{S}_t$ is normalized "column-wise" to the interval [0, 1]. Momentarily dropping the subscript t, this means computing the transformation

$$\mathbf{M}^s \mapsto \frac{\mathbf{M}^s - \mathbf{M}_{\min}}{\mathbf{M}_{\max} - \mathbf{M}_{\min}} \qquad \mathbf{S}^s \mapsto \frac{\mathbf{S}^s - \mathbf{S}_{\min}}{\mathbf{S}_{\max} - \mathbf{S}_{\min}},$$

where $\mathbf{M}_{\max}, \mathbf{M}_{\min}$ (resp. $\mathbf{S}_{\max}, \mathbf{S}_{\min}$) are arrays containing the element-wise maxima and minima of $\mathbf{M}$ (resp. $\mathbf{S}$) over the sampling dimension $1 \le s \le N_t$ (e.g. $(M_{\max})_{ij} = \max_s M^s_{ij}$ for every relevant i, j), and the division is

performed element-wise. Once the training data has been normalized, the validation data $\mathbf{M}_v, \mathbf{S}_v$ is normalized in a similar fashion based on the arrays $\mathbf{M}_{max/min}, \mathbf{S}_{max/min}$ computed from the training set.

Once the relevant neural networks have been trained, performance is measured through the empirical relative ℓ_1, ℓ_2 reconstruction of the FOM solution $\mathbf{x}$ over the validation set. In particular, denoting the reconstructed solution by $\tilde{\mathbf{x}}$, the empirical relative reconstruction error is computed as

$$R\ell_i(\mathbf{x}, \tilde{\mathbf{x}}) = \frac{\sum_s \left\| \mathbf{x}^s - \tilde{\mathbf{x}}^s \right\|_i}{\sum_s \left\| \mathbf{x}^s \right\|_i},$$

where $1 \leq s \leq N_v$. Note that the reconstructed solution $\tilde{\mathbf{x}} = \mathbf{g} \circ \mathbf{h}(\mathbf{x})$ for the compression problem while $\tilde{\mathbf{x}} = \mathbf{g} \circ \hat{\mathbf{x}}(t, \boldsymbol{\mu})$ for the prediction problem. In addition to the reconstruction error, it is helpful to report the memory required to store the trained models as well as the average wall-clock time per training epoch. While the memory cost is a property of the network, note that the wall-clock time is hardware and implementation dependent with significant fluctuation throughout the course of training. On the other hand, it provides a useful comparison (along with the plots of the loss) to estimate the difference in computational cost across autoencoder architectures. Pseudocode for the training procedure is given in Algorithm 1. Note that an early stopping criterion is employed, so that training is stopped when the validation loss does not decrease over a predefined number of consecutive epochs. All experiments are performed using Python 3.8.10 running PyTorch 1.8.1 [25] on an early 2015 model MacBook Pro with 2.7 GHz Dual-Core Intel Core i5 processor and 8 GB of RAM. The implementation of GCN2 convolution is modified from source code found in the PyTorch Geometric package [26].

Algorithm 1: Network Training

Result: Optimized functions $\mathbf{g}, \mathbf{h}, \hat{\mathbf{x}}$.
Initialize: normalized arrays $\mathbf{M}_t \in \mathbb{R}^{N_t \times (n_\mu + 1)}$, $\mathbf{S}_t \in \mathbb{R}^{N_t \times N \times n_f}$, $\mathbf{M}_v \in \mathbb{R}^{N_v \times (n_\mu + 1)}$, $\mathbf{S}_v \in \mathbb{R}^{N_v \times N \times n_f}$, early stopping parameter $c \in \mathbb{N}$, N_B, n_b;
count $= 0$ `// stopping criterion`
loss $= 100$ `// comparison criterion`
while count $< c$ **do**
  Randomly (but consistently) shuffle $\mathbf{M}_t, \mathbf{S}_t$ into N_B batches $\mathbf{M}_{t,b}, \mathbf{S}_{t,b}$ of size n_b;
  for $1 \leq$ batch $\leq N_B$ **do**
    Compute the loss $L(\mathbf{x}, t, \boldsymbol{\mu})$ over the n_b samples $\mathbf{x} \in \mathbf{S}_{t,b}, (t, \boldsymbol{\mu}) \in \mathbf{M}_{t,b}$;
    Backpropagate corresponding gradient information;
    Update network parameters based on ADAM rules;
  end
  Compute and return $L_v = L(\mathbf{x}, t, \boldsymbol{\mu})$ over the N_v samples $\mathbf{x} \in \mathbf{S}_v, (t, \boldsymbol{\mu}) \in \mathbf{M}_v$;
  if $L_v <$ loss **then**
    loss $= L_v$;
    count $= 0$;
  else
    count++
  end
end

4.2. Parameterized 1-D inviscid Burger's equation

The first experiment considers the parameterized one-dimensional inviscid Burgers' equation, which is a common model for fluids whose motion can develop discontinuities. In particular, let $\boldsymbol{\mu} = (\mu_1 \ \mu_2)^\top$ be a vector of parameters and consider the initial value problem,

$$\begin{aligned} w_t + \frac{1}{2}\left(w^2\right)_x &= 0.02e^{\mu_2 x}, \\ w(a, t, \boldsymbol{\mu}) &= \mu_1, \\ w(x, 0, \boldsymbol{\mu}) &= 1, \end{aligned} \tag{7}$$

Table 1
Architecture of GCNN-based autoencoder. Note that $\beta_\ell = \log\left(1+\frac{\theta}{\ell}\right)$ for the C layers and $\beta_\ell = \log\left(1+\frac{\theta}{N_l+\ell}\right)$ for the TC layers.

Layer	Input size	Kernel size	α	θ	Output size	Activation
1-C	(N, n_f)	$n_f \times n_f$	0.2	1.5	(N, n_f)	ReLU
			.....			
N_l-C	(N, n_f)	$n_f \times n_f$	0.2	1.5	(N, n_f)	ReLU
Samples of size (N, n_f) are flattened to size $(n_f * N)$.						
1-FC	$n_f * N$				n	Identity
End of encoding layers. Beginning of decoding layers.						
2-FC	n				$n_f * N$	Identity
Samples of size $(n_f * N)$ are reshaped to size (N, n_f).						
1-TC	(N, n_f)	$n_f \times n_f$	0.2	1.5	(N, n_f)	ReLU
			.....			
N_l-TC	(N, n_f)	$n_f \times n_f$	0.2	1.5	(N, n_f)	ReLU
End of decoding layers. Prediction layers listed below.						
3-FC	$n_\mu + 1$				50	ELU
4-FC	50				50	ELU
			.....			
10-FC	50				n	Identity

where subscripts denote partial differentiation and $w = w(x, t, \boldsymbol{\mu})$ represents the conserved quantity of the fluid for $x \in [a, b]$. As a first test, it is interesting to examine how the CNN, GCNN, and FCNN autoencoder-based ROMs perform on this relatively simple hyperbolic equation when compared to linear techniques such as POD. First, note that this problem is easily converted to the form (1) by discretizing the interval $[a, b]$ with finite differences, in which case the solution at each nodal point is $w^i(t, \boldsymbol{\mu}) := w(x_i, t, \boldsymbol{\mu})$, and $\mathbf{w}(t, \boldsymbol{\mu}) \in \mathbb{R}^N$ is a vector for each $(t, \boldsymbol{\mu})$ of length equal to the number N of nodes. With this, the Eqs. (7) can be discretized and solved for various parameter values using a simple upwind forward Euler scheme. In particular, given stepsizes Δx, Δt, let w_k^i denote the solution $\mathbf{w}$ at time step k and spatial node i. Then, simple forward differencing gives

$$\frac{w_{k+1}^i - w_k^i}{\Delta t} + \frac{1}{2}\frac{\left(w^2\right)_k^i - \left(w^2\right)_k^{i-1}}{\Delta x} = 0.02e^{\mu_2 x},$$

which expresses the value w^i at $t = (k+1)\Delta t$ in terms of its value at $t = k\Delta t$. For the experiment shown, the discretization parameters chosen are $\Delta x = 100/256$ for $x \in [0, 100]$ and $\Delta t = 1/10$ for $t \in [0, 10]$. The parameters $\boldsymbol{\mu}$ used for training are $\{(2+0.5i, 0.015+0.005j)\}$ where $0 \le i \le 2$ and $0 \le j \le 3$, while the validation parameters are the lattice midpoints $\{(2.25 + 0.5i, 0.0175 + 0.005j)\}$ where $0 \le i \le 1$ and $0 \le j \le 2$. A precise description of the autoencoder architectures employed can be found in Tables 1, 2, and 3, and a precise description of the POD-ROM employed can be found in the Appendix. The initial learning rates for the ADAM optimizer are chosen as 0.0025 for in the GCNN case, 0.001 in the CNN case, and 0.0005 in the FCNN case. The early stopping criterion for this example is 200 epochs and the mini-batch size is 20.

As seen in Table 4, the results for both the prediction and compression problems are significantly different across ROM architectures and latent space sizes. Indeed, the CNN autoencoder is consistently the most accurate on the prediction problem when n is small while also being the most time consuming and memory consumptive, and its performance does not improve with increased latent dimension. This is consistent with prior work demonstrating that nonlinear ROMs can outperform linear techniques such as POD when n is close to the theoretical minimum. Conversely, the quality of the POD-ROM approximation continues to improve as more modes are added and eventually surpasses the performance of all network-based architectures. This is not surprising, as the POD-ROM is not strictly data-driven like the neural network ROMs and so benefits from additional structure. Note that the

Table 2
FCNN architecture for the 1-D Burgers' example.

Layer	Input size	Output size	Activation
Samples of size (256,1) are flattened to size (256).			
1-FC	256	64	ELU
1-BN	64	64	Identity
2-FC	64	n	ELU
End of encoding layers. Beginning of decoding layers.			
3-FC	n	64	ELU
2-BN	64	64	Identity
4-FC	64	256	ELU
Samples of size (256) are reshaped to size (256,1)			
End of decoding layers. Prediction layers listed below.			
5-FC	$n_\mu + 1$	50	ELU
6-FC	50	50	ELU
7-FC	50	50	ELU
8-FC	50	n	ELU

Table 3
CNN autoencoder architecture for the 1-D Burgers' example, identical to [7, Table 1,2]. Prediction layers (not pictured) are the same as in the FCNN case (c.f. Table 2).

Layer	Input size	Kernel size	Stride	Padding	Output size	Activation
Samples of size (256,1) are reshaped to size (1, 16, 16).						
1-C	(1, 16, 16)	5×5	1	SAME	(8, 16, 16)	ELU
2-C	(8, 16, 16)	5×5	2	SAME	(16, 8, 8)	ELU
3-C	(16, 8, 8)	5×5	2	SAME	(32, 4, 4)	ELU
4-C	(32, 4, 4)	5×5	2	SAME	(64, 2, 2)	ELU
Samples of size (64, 2, 2) are flattened to size (256).						
1-FC	256				n	ELU
End of encoding layers. Beginning of decoding layers.						
2-FC	n				256	ELU
Samples of size (256) are reshaped to size (64, 2, 2).						
1-TC	(64, 2, 2)	5×5	2	SAME	(64, 4, 4)	ELU
2-TC	(64, 4, 4)	5×5	2	SAME	(32, 8, 8)	ELU
3-TC	(32, 8, 8)	5×5	2	SAME	(16, 16, 16)	ELU
4-TC	(16, 16, 16)	5×5	1	SAME	(1, 16, 16)	ELU
Samples of size (1, 16, 16) are reshaped to size (256, 1).						

performance of the GCNN-ROM increases quite a bit when the latent dimension is increased from 3 to 10, but remains significantly less accurate than the ROM based on CNN or FCNN. Interestingly, on the prediction problem the performance of all networks appears to be bottlenecked by the fully connected network $\hat{\mathbf{x}}$ which simulates the low-dimensional dynamics, as their accuracy does not increase when the latent dimension increases from $n = 10$ to $n = 32$. This suggests that a ROM based on Newton or quasi-Newton methods may produce better results in this case.

On the compression problem, the results show that all ROM architectures benefit from an increased latent space dimension when the goal is to encode and decode, with the GCNN autoencoder benefiting the most and the CNN autoencoder benefiting the least. Again, the GCNN architecture struggles when the latent space dimension is set to the theoretical minimum $n = 3$, however its performance exceeds CNN and becomes competitive with that of FCNN once the latent dimension is increased to $n = 32$, at half of the memory cost. However, at this size the linear POD approximation is also quite good, nearly matching the neural network approximations in quality.

Table 4
Numerical results for the 1-D parameterized Burgers' example corresponding to the prediction problem (left) and the compression problem (right).

Method	Encoder/Decoder + Prediction					Encoder/Decoder only				
	n	$R\ell_1\%$	$R\ell_2\%$	Size (kB)	Time per epoch (s)	n	$R\ell_1\%$	$R\ell_2\%$	Size (kB)	Time per epoch (s)
POD	3	3.67	8.50	6.27	N/A	3	3.56	8.29	6.27	N/A
GCNN		4.41	8.49	79.0	0.55		2.54	5.31	13.1	0.46
CNN		0.304	0.605	965	2.2		0.290	0.563	953	2.2
FCNN		1.62	3.29	167	0.28		0.658	1.66	144	0.22
POD	10	1.75	3.63	20.6	N/A	10	1.18	2.86	20.6	N/A
GCNN		2.08	3.73	94.8	0.58		0.706	1.99	27.4	0.47
CNN		0.301	0.630	992	2.4		0.215	0.409	967	2.2
FCNN		0.449	1.15	172	0.27		0.171	0.361	148	0.23
POD	32	0.137	0.264	65.7	N/A	32	0.117	0.249	65.7	N/A
GCNN		2.59	4.17	145	0.59		0.087	0.278	72.6	0.46
CNN		0.350	0.675	1040	2.3		0.216	0.384	1010	2.2
FCNN		0.530	1.303	188	0.27		0.098	0.216	159	0.23

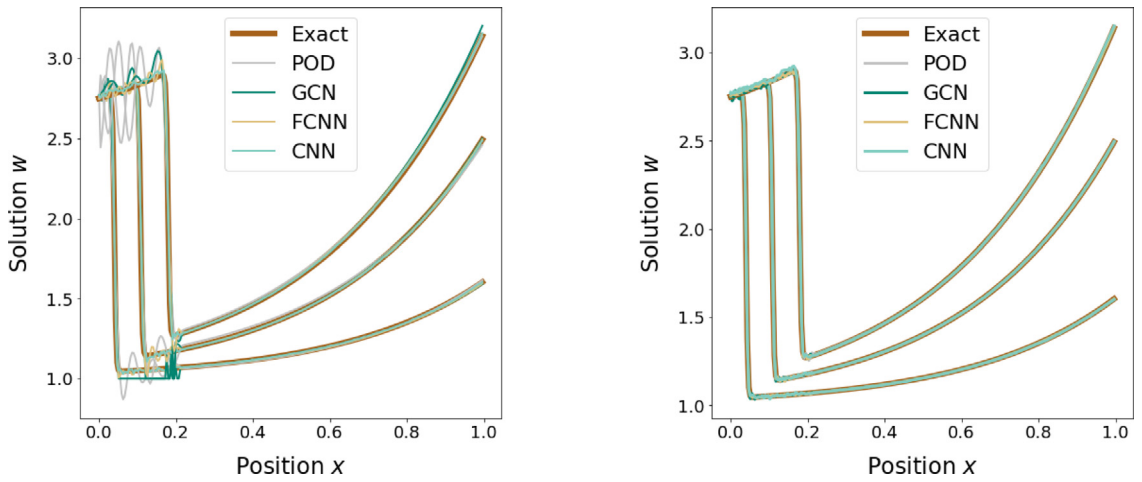

Fig. 3. Plots corresponding to the Burgers' example with $\boldsymbol{\mu} = \begin{pmatrix}0.275 & 0.0275\end{pmatrix}^\top$ and $t = 0.4, 1.1, 1.8$. Left: results for prediction problem with $n = 10$. Right: results for compression problem with $n = 32$.

Remark 4.1. Instead of increasing the latent dimension n, it is reasonable to consider increasing the depth of the autoencoder and prediction networks. However, we find that increasing the depth of the GCNN autoencoder beyond $N_l = 4$ does not significantly improve its performance on either the prediction or compression problems. On the other hand, the GCNN-ROM benefits modestly from an increase in depth of the prediction network from 4 to 8 layers, although the CNN and FCNN based ROMs do not. Therefore, all prediction experiments use 8 fully connected layers in the GCNN case and 4 in the CNN and FCNN cases.

Fig. 3 illustrates these results at various values of t for the parameter $\mu = \begin{pmatrix}2.75 & 0.0275\end{pmatrix}^\top$. Clearly, the architecture based on CNN is best equipped for solving the prediction problem in this case. On the other hand, when $n = 32$ it is evident that all architectures can compress and reconstruct solution data well, although the FCNN and GCNN yield the best results. Fig. 4 shows the corresponding plots of the losses incurred during network training. Note that all architectures have comparable training and validation losses for the compression problem, while all but the GCNN overfit the data slightly on the prediction problem.

It is also worthwhile to compare how the offline and online costs of each ROM architecture scale with increasing degrees of freedom. Table 5 displays one such comparison using data from the present example. Given the parameter $\mu = \begin{pmatrix}2.75 & 0.0275\end{pmatrix}^\top$, $N_s = 1200$ FOM snapshots are generated at the resolutions $N = 256, 1024, 4096$, which

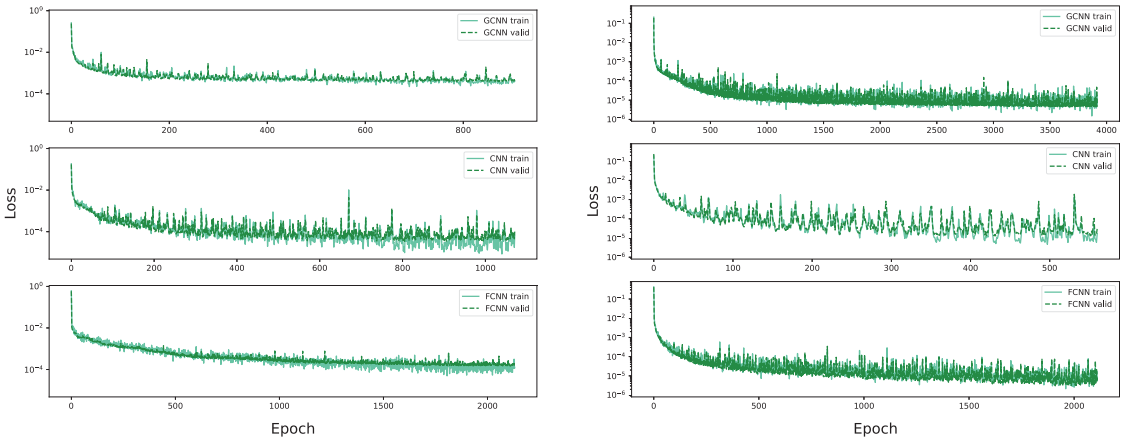

Fig. 4. Training and validation losses incurred during neural network training for the 1-D Burgers' example. Left: prediction problem with $n = 10$. Right: compression problem with $n = 32$.

Table 5
Offline and online costs (in seconds) of evaluating the ROMs corresponding to the 1-D parameterized Burgers' example with $n = 10$.

		N					
		256		1024		4096	
		Offline	Online	Offline	Online	Offline	Online
Method	POD	0.1226	0.02675	0.8504	0.02836	15.22	0.07285
	GCNN	187.7	0.01383	385.9	0.04159	3243	0.1448
	CNN	530.5	0.1625	1830	1.171	4494	3.491
	FCNN	147.2	0.002186	322.5	0.01053	1339	0.06497

are used to evaluate the computation time of each ROM in seconds. The offline and online costs reported in Table 5 represent respectively the total time required to train each ROM and the time required to evaluate each trained ROM. Note that $n = 10$ is used, the offline cost of the POD-ROM includes the assembly time for the low-dimensional system, and the early stopping criterion for the neural networks is relaxed to 50 epochs. Here and in Fig. 5 it can be seen that none of the nonlinear ROMs can match the speed of POD offline, although both the GCNN and FCNN architectures are more efficient than POD online when the dimension of the problem is small enough. Conversely, for this example POD scales better with increasing n and eventually becomes more efficient in both categories. Interestingly, the highest performing CNN architecture is seen to be the slowest both offline and online by a large margin, potentially because of its much larger memory size.

4.3. Parameterized 2-D heat equation

The next experiment involves the parameterized two-dimensional heat equation on a uniform grid. In particular, let $\Omega = [0, 1] \times [0, 2]$, $t \in [0, 4]$, $\boldsymbol{\mu} \in [0.1, 0.5] \times \left[\frac{\pi}{2}, \pi\right]$ and consider the problem

$$\begin{aligned} &u_t - \Delta u = 0, \\ &u(0, y, t) = -0.5, \\ &u(1, y, t) = \mu_1 \cos(\mu_2 y), \\ &u(x, y, 0) = 0, \end{aligned} \tag{8}$$

where $u = u(x, y, t, \boldsymbol{\mu})$ is the temperature profile. Denoting the L^2 inner product on Ω by $(\cdot, \cdot)$, multiplying the first equation by a test function $v \in H_0^1(\Omega)$ and integrating by parts yields the standard weak equation

$$(u_t, v) + (\nabla u, \nabla v) = 0, \qquad \text{for all } v \in H_0^1(\Omega),$$

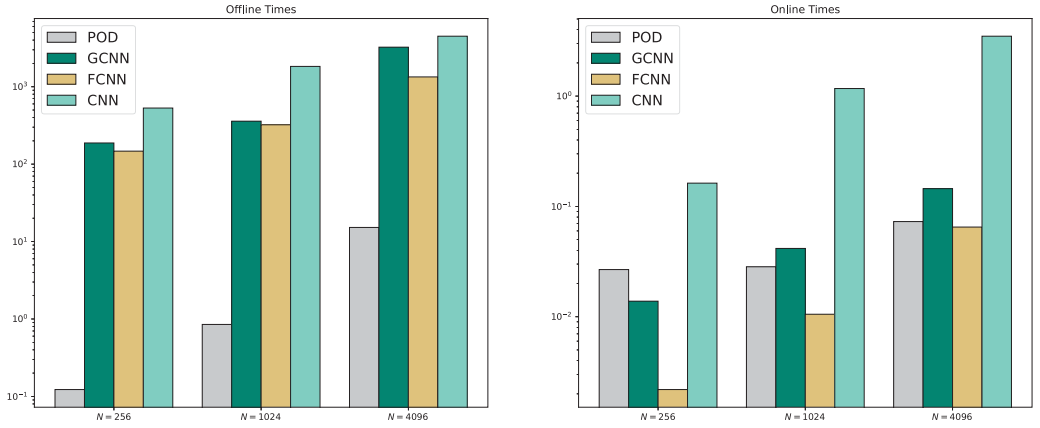

Fig. 5. Plots of the computational costs in Table 5 corresponding to the 1-D parameterized Burgers' example.

which can be discretized and solved using standard finite element techniques. In particular, choosing a finite element basis $\{\varphi_i\}$ so that the discretized solution has coefficient vector $\mathbf{u}$ yields the linear system

$$\mathbf{M}\mathbf{u}_t = -\mathbf{S}\mathbf{u},$$

where $\mathbf{M} = (m_{ij})$, $\mathbf{S} = (s_{ij})$ are the symmetric mass and stiffness matrices with entries

$$m_{ij} = \left(\varphi_i, \varphi_j\right), \qquad s_{ij} = \left(\nabla\varphi_i, \nabla\varphi_j\right). \tag{9}$$

The present experiment examines the ability of the neural network ROMs to predict solutions to this parameterized boundary value problem. In the full order case, u is semi-discretized using bilinear Q_1-elements over a 32×64 grid to yield $\mathbf{u}(t, \boldsymbol{\mu})$ of dimension $N = 2145$, and the IFISS software package [27] is used to assemble and solve the boundary value problem (8) using a backward Euler method with stepsize $\Delta t = 1/100$. The system is integrated for a time period of 4 s at various parameters $\boldsymbol{\mu}$. Since no classical solution to (8) exists at $t = 0$, solution snapshots are stored starting from $t = 0.01$. Some solutions along this heat flow contained in the validation set are displayed in Fig. 7.

The parameters $\boldsymbol{\mu}$ appearing in the boundary conditions are again chosen to lie in a lattice. In particular, the 9 values used for training are $\{(0.1 + 0.2i, \frac{\pi}{2} + \frac{\pi}{4}j)\}$ for $0 \le i, j \le 2$ and the 4 validation values are the lattice midpoints $\{(0.2+0.2i, \frac{3\pi}{8} + \frac{\pi}{2}j)\}$. A sample from the validation set along with its ROM reconstructions for the case $n = 10$ is shown in Fig. 7. The precise architectures used are displayed in Tables 1, 6, and 7. The initial learning rates are 2.5×10^{-3} for the GCNN case, 5×10^{-4} for the CNN case, and 5×10^{-4} for the FCNN case. The early stopping criterion for network training is 100 epochs, and the mini-batch size is $n_b = 32$.

The results for this experiment are displayed in Table 8, which clearly illustrates the drawbacks of using the standard CNN architecture along with solution reshaping. Not only is the computation time increased due to the introduction of almost 2000 fake nodes, but the accuracy of the CNN-ROM is significantly lower than the FCNN-ROM despite its comparable or larger memory cost. On the other hand, the architecture based on GCNN is both the least memory consumptive and the most accurate on each problem when $n \ge 10$. Again, results show that the GCNN-ROM is most effective on the compression problem, demonstrating significantly improved accuracy when the latent dimension is not too small. Conversely, if a latent dimension equal to the theoretical minimum is desired, then the ROM based on deep CNN is still superior, potentially due to its multiple levels of filtration with relatively large kernels. An example ROM reconstruction along with pointwise error plots is displayed in Fig. 8. Interestingly, the loss plots in Fig. 6 show that all autoencoder networks exhibit significant overfitting on this dataset, suggesting that none of the models has sufficient generalizability to produce optimal results on new data.

4.4. Unsteady 2-D Navier–Stokes equations

The final experiment considers the unsteady Navier–Stokes equations on a rectangular domain Ω with a slightly off-center cylindrical hole, which is a common benchmark model for computational fluid dynamics (c.f [28]). The

Table 6
FCNN autoencoder architecture for the 2-D heat example.

Layer	Input size	Output size	Activation
Samples of size (2145,1) are flattened to size (2145).			
1-FC	2145	215	ELU
1-BN	215	215	Identity
2-FC	215	n	ELU
End of encoding layers. Beginning of decoding layers.			
3-FC	n	215	ELU
2-BN	215	215	Identity
4-FC	215	2145	ELU
Samples of size (2145) are reshaped to size (2145,1)			
End of decoding layers. Prediction layers listed below.			
5-FC	$n_\mu + 1$	50	ELU
6-FC	50	50	ELU
7-FC	50	50	ELU
8-FC	50	n	ELU

Table 7
CNN autoencoder architecture for the 2-D heat example. Prediction layers (not pictured) are the same as in the FCNN case (c.f. Table 6).

Layer	Input size	Kernel size	Stride	Padding	Output size	Activation
Samples of size (2145, 1) are zero padded to size (4096,1) and reshaped to size (1, 64, 64).						
1-C	(1, 64, 64)	5 × 5	2	SAME	(4, 32, 32)	ELU
2-C	(4, 32, 32)	5 × 5	2	SAME	(16, 16, 16)	ELU
3-C	(16, 16, 16)	5 × 5	2	SAME	(64, 8, 8)	ELU
4-C	(64, 8, 8)	5 × 5	2	SAME	(256, 4, 4)	ELU
Samples of size (256, 4, 4) are flattened to size (4096).						
1-FC	4096				n	ELU
End of encoding layers. Beginning of decoding layers.						
2-FC	n				4096	ELU
Samples of size (4096) are reshaped to size (256, 4, 4).						
1-TC	(256, 4, 4)	5 × 5	2	SAME	(64, 8, 8)	ELU
2-TC	(64, 8, 8)	5 × 5	2	SAME	(16, 16, 16)	ELU
3-TC	(16, 16, 16)	5 × 5	2	SAME	(4, 32, 32)	ELU
4-TC	(4, 32, 32)	5 × 5	2	SAME	(1, 64, 64)	ELU
Samples of size (1, 64, 64) are reshaped to size (4096, 1) and truncated to size (2145, 1).						

domain Ω is pictured in Fig. 9, and the governing system is

$$\begin{aligned}
&\mathbf{u}_t - \nu \Delta \mathbf{u} + (\mathbf{u} \cdot \nabla)\,\mathbf{u} + \nabla p = 0,\\
&\nabla \cdot \mathbf{u} = 0,\\
&\mathbf{u}|_{t=0} = 0,
\end{aligned}$$

subject to the boundary conditions

$$\begin{aligned}
\mathbf{u} &= 0 \qquad && \text{on } \Gamma_2, \Gamma_4, \Gamma_5,\\
\nu(\mathbf{n} \cdot \nabla \mathbf{u}) - p\mathbf{n} &= 0 \qquad && \text{on } \Gamma_3,\\
\mathbf{u} &= \begin{pmatrix} \frac{6y(0.41-y)}{0.41^2} & 0 \end{pmatrix}^\top \qquad && \text{on } \Gamma_1,
\end{aligned}$$

Table 8
Numerical results for the 2-D parameterized heat equation corresponding to the prediction problem (left) and the compression problem (right).

Network	Encoder/Decoder + Prediction					Encoder/Decoder only				
	n	$R\ell_1\%$	$R\ell_2\%$	Size (MB)	Time per epoch (s)	n	$R\ell_1\%$	$R\ell_2\%$	Size (MB)	Time per epoch (s)
GCNN		7.19	9.21	0.132	9.5		6.96	9.21	0.0659	9.2
CNN	3	3.26	4.58	3.64	18	3	3.36	3.81	3.62	18
FCNN		4.75	6.19	3.74	3.3		4.22	5.69	3.72	3.1
GCNN		2.87	3.82	0.253	9.6		2.06	2.85	0.186	9.4
CNN	10	3.07	4.38	3.87	18	10	2.45	2.90	3.85	18
FCNN		2.96	3.97	3.76	3.3		2.32	2.92	3.73	3.1
GCNN		2.55	3.48	0.636	9.6		1.05	1.91	0.564	9.2
CNN	32	2.30	3.73	4.60	19	32	2.34	2.91	4.57	18
FCNN		2.65	4.25	3.80	3.2		1.61	2.31	3.77	3.2

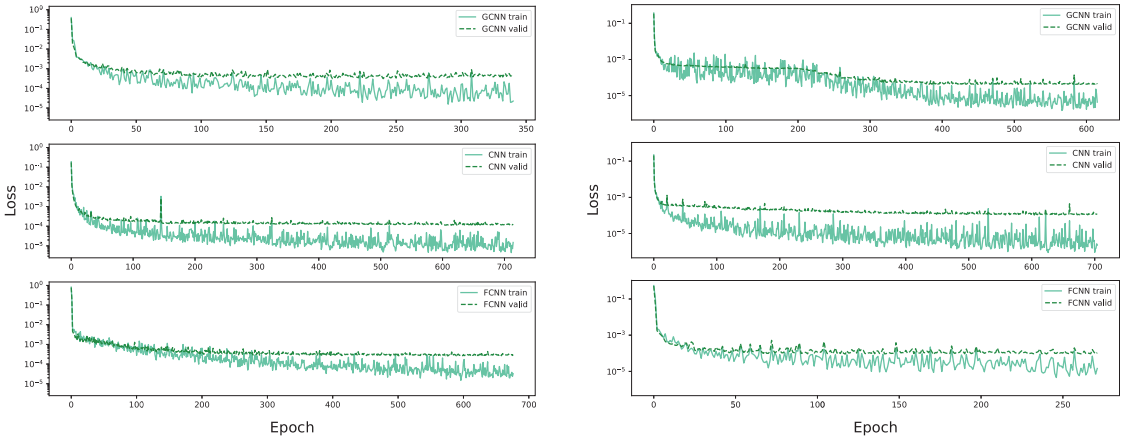

Fig. 6. Training and validation losses incurred during neural network training for the heat example. Left: prediction problem with $n = 32$. Right: compression problem with $n = 32$.

where $(\mathbf{u}, p)$ are the velocity and pressure functions and ν is the viscosity parameter. Recall the Reynolds number

$$\mathrm{Re} = \frac{|\mathbf{u}|_{mean} L}{\nu},$$

where $|\mathbf{u}|_{mean}$ is the average speed along the flow and L is the characteristic length of the flow configuration. The goal of this experiment is to see how well the ROM-autoencoder architectures can compress and predict solutions to these equations for a given parameter pair (t, Re) when compared to POD and to each other. As the meshed domain is highly irregular, this represents a good test case for the GCNN-ROM introduced before.

To generate the training and validation data, three values of ν are selected corresponding to $(5 + i) \times 10^{-4}$ for $0 \leq i \leq 2$. This yields solutions with corresponding $\mathrm{Re} \in \{214, 180, 155\}$. The flow is simulated starting from rest over the time interval $[0, 1]$, using Netgen/NGSolve with P_3/P_2 Taylor–Hood finite elements for spatial discretization and implicit–explicit Euler with $\Delta t = 5 \times 10^{-5}$ for time stepping. Solution snapshots are stored once every 50 time steps, yielding a total of 400 snapshots per example. Note that the domain Ω is represented by an unstructured irregular mesh whose element density is higher near the cylindrical hole, yielding two solution features (the x, y components of velocity) with dimension $N = 10104$ equal to the number of nodes. For the present experiments, the data corresponding to $\mathrm{Re} = 155, 214$ are used to train the ROMs and the data corresponding to $\mathrm{Re} = 180$ are used for validation. The network architectures used in this case are given in Tables 1, 9, and 10, respectively. The POD-ROM used is detailed in the Appendix. The early stopping criterion is 50 epochs, and the initial learning rate used to train each network is 5×10^{-4}.

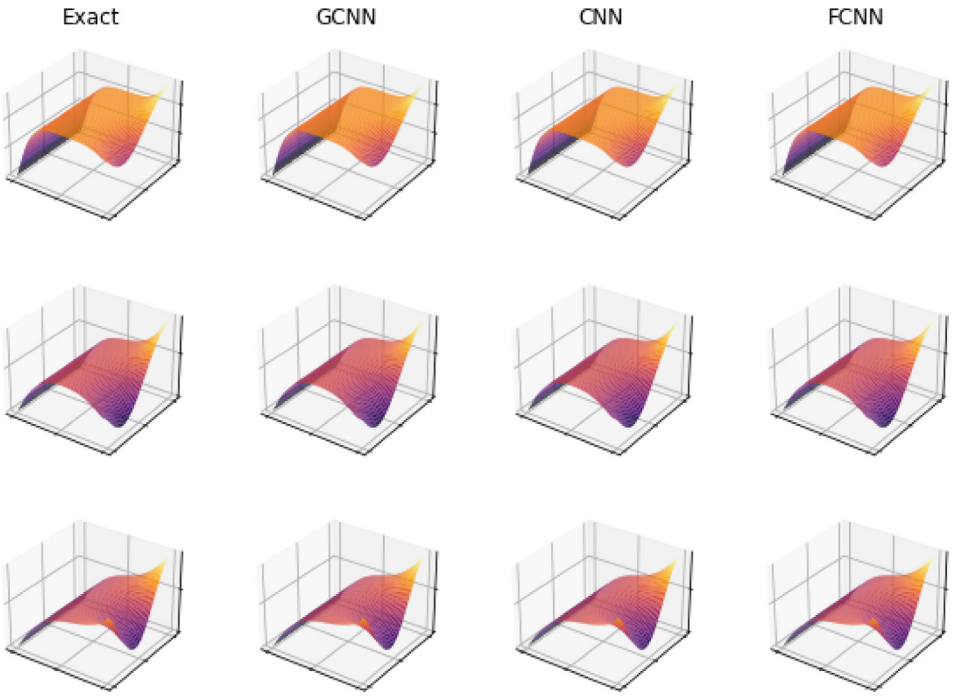

Fig. 7. Example solutions to the 2-D parameterized heat equation contained in the validation set and their predictions given by the neural networks corresponding to $n = 10$. The parameters $(t, \boldsymbol{\mu})$ are: $\left(1, 0.2, \frac{5\pi}{8}\right)$ (top), $\left(2, 0.4, \frac{5\pi}{8}\right)$ (mid), $\left(3, 0.4, \frac{7\pi}{8}\right)$ (bottom).

Table 9
FCNN autoencoder architecture for the Navier–Stokes example.

Layer	Input size	Output size	Activation
Samples of size (10 104, 2) are flattened to size (20 208).			
1-FC	20 208	2021	ELU
1-BN	2021	2021	Identity
2-FC	2021	202	ELU
2-BN	202	202	Identity
3-FC	202	n	ELU
End of encoding layers. Beginning of decoding layers.			
4-FC	n	202	ELU
3-BN	202	202	Identity
5-FC	202	2021	ELU
4-BN	2021	2021	Identity
6-FC	2021	20 208	ELU
Samples of size (20 208) are reshaped to size (10 104, 2)			
End of decoding layers. Prediction layers listed below.			
7-FC	$n_\mu + 1$	50	ELU
8-FC	50	50	ELU
9-FC	50	50	ELU
10-FC	50	n	ELU

The results of this experiment are given in Table 11, making it clear that the trick of reshaping the nodal features in order to apply CNN carries an associated accuracy cost. Indeed, the performance of the CNN-ROM on the prediction problem is inferior to both the GCNN-ROM and FCNN-ROM (provided the latent space is not too small) despite its much higher computational cost. Moreover, on this highly irregular domain the CNN-based architecture

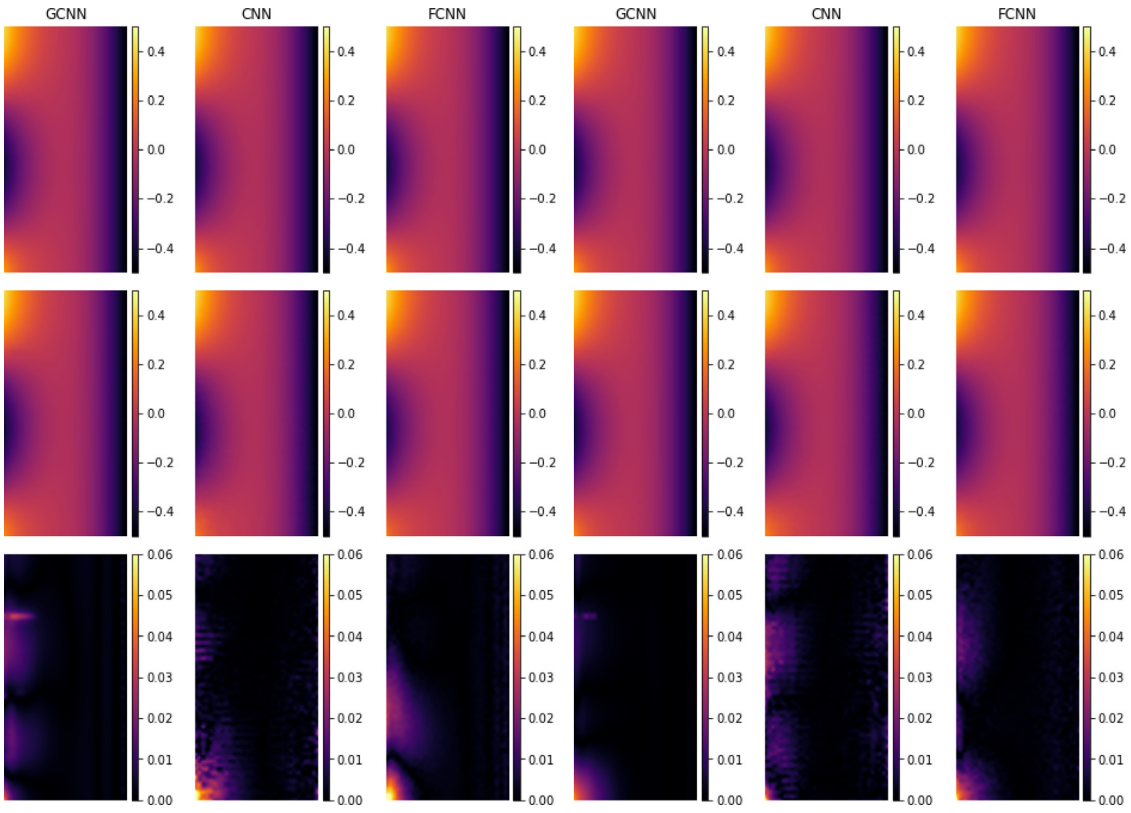

Fig. 8. Solutions corresponding to the parameter $(t, \boldsymbol{\mu}) = \begin{pmatrix} 3 & 0.4 & \frac{7\pi}{8} \end{pmatrix}^\top$ on the prediction problem with $n = 10$ (left) and on the compression problem with $n = 32$ (right).

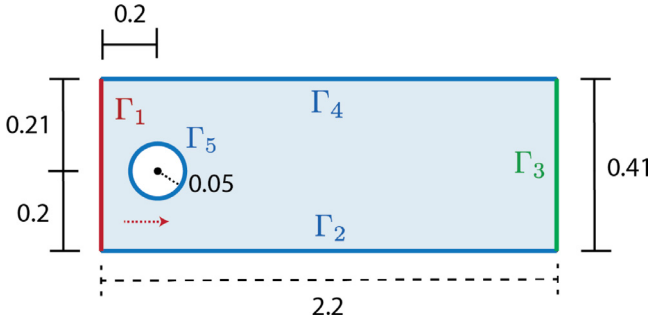

Fig. 9. The domain Ω used in the Navier–Stokes example.

does not even offer benefits in the low-dimensional $n = 2$ case, where the architecture based on FCNN is roughly 5 times faster to train while maintaining superior accuracy. It is also apparent that the network-based ROMs behave differently than the POD-ROM in this case. The FCNN-ROM is consistently the most accurate throughout on the prediction problem, with this difference being the most noticeable when the dimension n is very small. On the other hand, the POD-ROM approximation is very competitive when $n = 32$, but degrades with larger n while the network approximations continue to improve. This is perhaps due to the fact that 99.97 of the total squared POD singular values are captured here with only 32 modes 93.55% with 2 modes), limiting the benefit to increasing n further.

On the other hand, there are still interesting aspects of the network ROMs noticeable here. As before, the GCNN-ROM struggles when n is small but gains in performance as the latent dimension increases. In fact, results indicate

Table 10
CNN autoencoder architecture for the Navier–Stokes example. Prediction layers (not pictured) are the same as in the FCNN case (c.f. Table 9).

Layer	Input size	Kernel size	Stride	Padding	Output size	Activation
Samples of size (10 104, 2) are zero padded to size (16 384,2) and reshaped to size (2, 128, 128).						
1-C	(2, 128, 128)	5 × 5	2	SAME	(8, 64, 64)	ELU
2-C	(8, 64, 64)	5 × 5	2	SAME	(32, 32, 32)	ELU
3-C	(32, 32, 32)	5 × 5	2	SAME	(128, 16, 16)	ELU
4-C	(128, 16, 16)	5 × 5	2	SAME	(512, 8, 8)	ELU
5-C	(512, 8, 8)	5 × 5	2	SAME	(2048, 4, 4)	ELU
Samples of size (2048, 4, 4) are flattened to size (2∗16384).						
1-FC	32 768				n	ELU
End of encoding layers. Beginning of decoding layers.						
2-FC	n				32 768	ELU
Samples of size (2∗16384) are reshaped to size (2048, 4, 4).						
1-TC	(2048, 4, 4)	5 × 5	2	SAME	(512, 8, 8)	ELU
2-TC	(512, 8, 8)	5 × 5	2	SAME	(128, 16, 16)	ELU
3-TC	(128, 16, 16)	5 × 5	2	SAME	(32, 32, 32)	ELU
4-TC	(32, 32, 32)	5 × 5	2	SAME	(8, 64, 64)	ELU
5-TC	(8, 64, 64)	5 × 5	2	SAME	(2, 128, 128)	ELU
Samples of size (2, 128, 128) are reshaped to size (16 384, 2) and truncated to size (10 104, 2).						

Table 11
Numerical results for the Navier–Stokes example corresponding to the prediction problem (left) and the compression problem (right). Note that the relative errors for the POD-ROM are computed with respect to the inner product defined by the finite element mass matrix.

Network	Encoder/Decoder + Prediction					Encoder/Decoder only				
	n	$R\ell_1\%$	$R\ell_2\%$	Size (MB)	Time per epoch (s)	n	$R\ell_1\%$	$R\ell_2\%$	Size (MB)	Time per epoch (s)
POD	2	10.0	15.9	0.323	N/A	2	6.75	10.0	0.323	N/A
GCN		9.07	14.6	0.476	33		7.67	12.2	0.410	32
CNN		7.12	11.1	224	210		11.2	17.6	224	190
FCNN		1.62	2.87	330	38		1.62	2.70	330	38
POD	32	2.70	3.97	5.17	N/A	32	0.428	0.674	5.17	N/A
GCN		2.97	5.14	5.33	32		0.825	1.49	5.26	32
CNN		4.57	7.09	232	230		4.61	7.24	232	220
FCNN		1.39	2.64	330	38		0.680	1.12	330	38
POD	64	2.80	4.36	10.3	N/A	64	0.191	0.318	10.3	N/A
GCN		2.88	4.96	10.5	33		0.450	0.791	10.4	33
CNN		3.42	5.33	241	270		2.42	3.57	241	260
FCNN		1.45	2.64	330	38		0.704	1.19	330	37

that the GCNN is superior to the FCNN on the compression problem when $n = 64$ while also being faster to train and an order of magnitude less memory consumptive. Although $n = 64$ is far from the "best case scenario" of $n = 2$, it is still much smaller than the original dimension of $N = 10104$ and may be useful in a real-time or many-query setting. Interestingly, Fig. 10 shows that the CNN-ROM does not overfit on the prediction or compression problems, indicating that the standard CNN may not be expressive enough to handle complicated irregular data that has been aggressively reshaped. Similarly, the GCNN-ROM does not overfit during the compression examples, although its performance scales with the dimension of the latent space, offering predictive accuracy near that of the FCNN with a memory cost near that of POD. Figs. 11, 12, 13, and 14 provide a visual comparison of the ROM reconstructions and their relative errors at $t = 0.99$. Finally, note that for this problem it may be infeasible in the case $n = 2$ to generate an accurate predictive ROM using any method, since it is still an open question whether the mapping $(t, \mathrm{Re}) \mapsto \mathbf{u}$ is unique in this case.

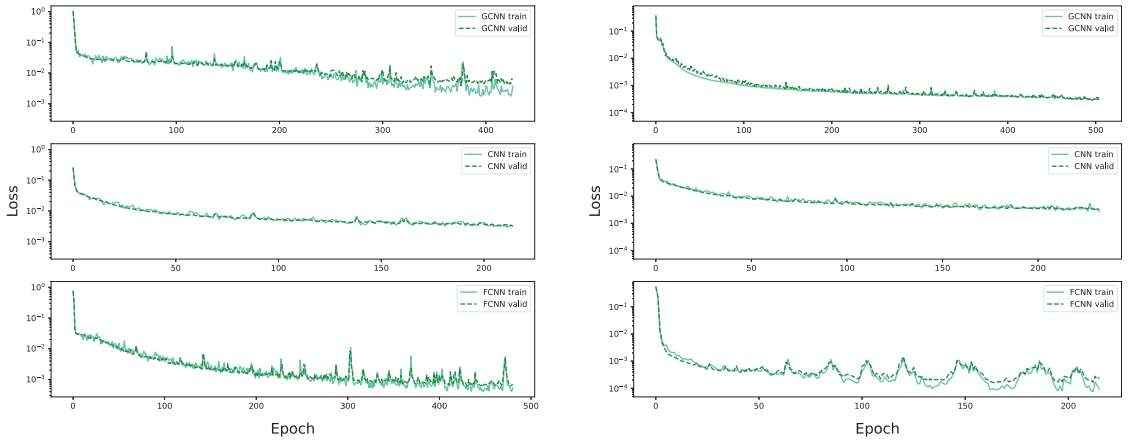

Fig. 10. Training and validation losses incurred during neural network training for the Navier–Stokes example. Left: prediction problem with $n = 32$. Right: compression problem with $n = 32$.

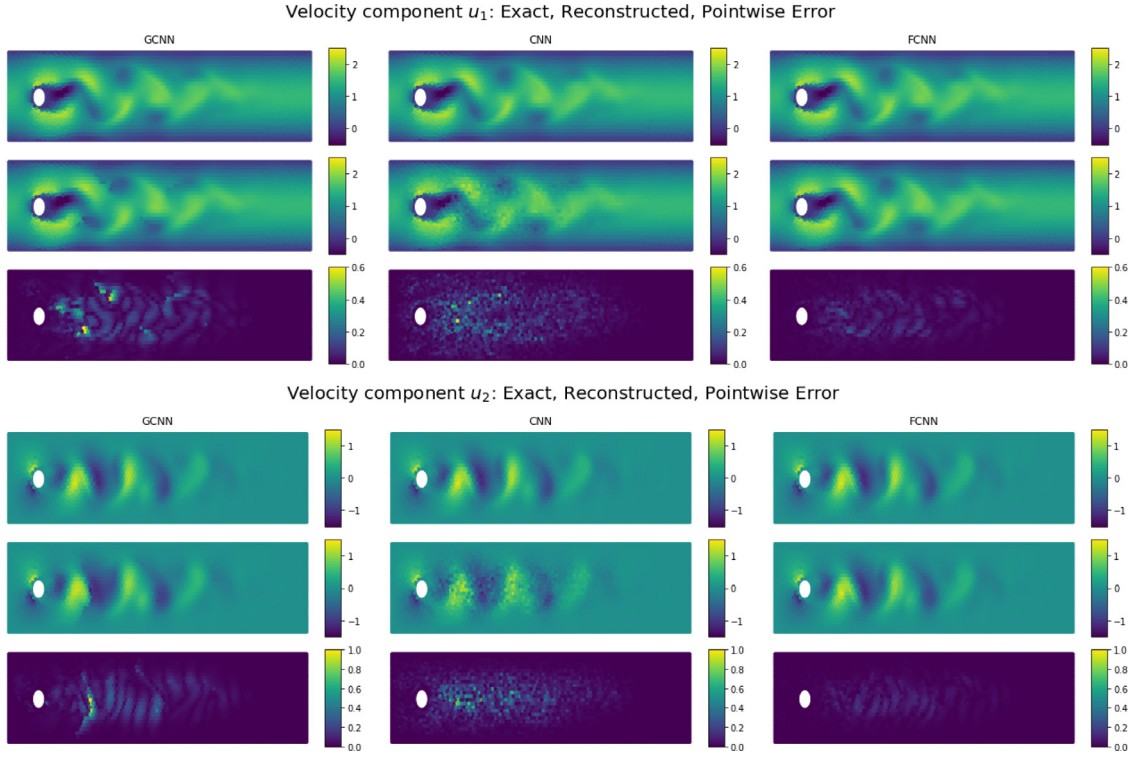

Fig. 11. Solutions to the prediction problem with $n = 32$ for the parameter $(t, \boldsymbol{\mu}) = \begin{pmatrix} 0.99 & 180 \end{pmatrix}^\top$.

5. Conclusion

Neural network-based strategies for the reduced-order modeling of PDEs have been discussed, and a novel autoencoder architecture based on graph convolution has been proposed for this purpose. Through benchmark problems, the performance of three fundamentally different autoencoder-based ROM architectures has been compared, demonstrating that each architecture has advantages and disadvantages depending on the task involved and the connectivity pattern of the input data. In general, results indicate that the proposed graph convolutional ROM provides an accurate and lightweight alternative to fully connected ROMs on unstructured finite element data, greatly

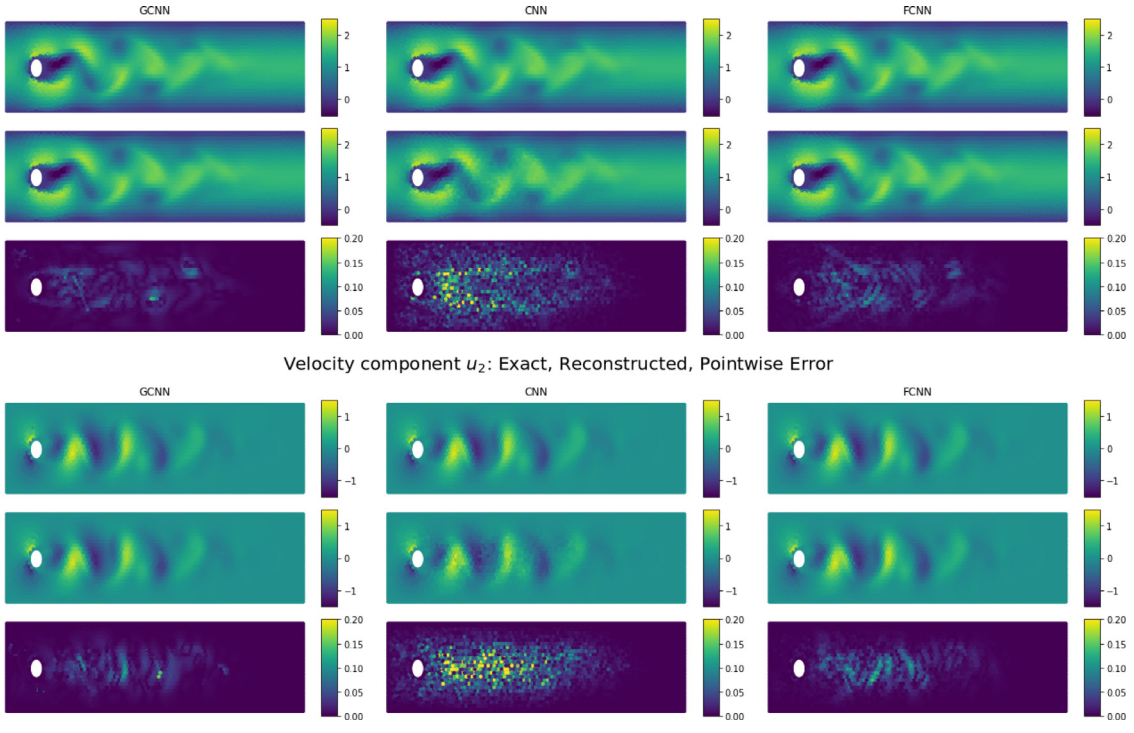

Fig. 12. Solutions to the compression problem with $n = 64$ for the parameter $(t, \boldsymbol{\mu}) = \begin{pmatrix} 0.99 & 180 \end{pmatrix}^\top$.

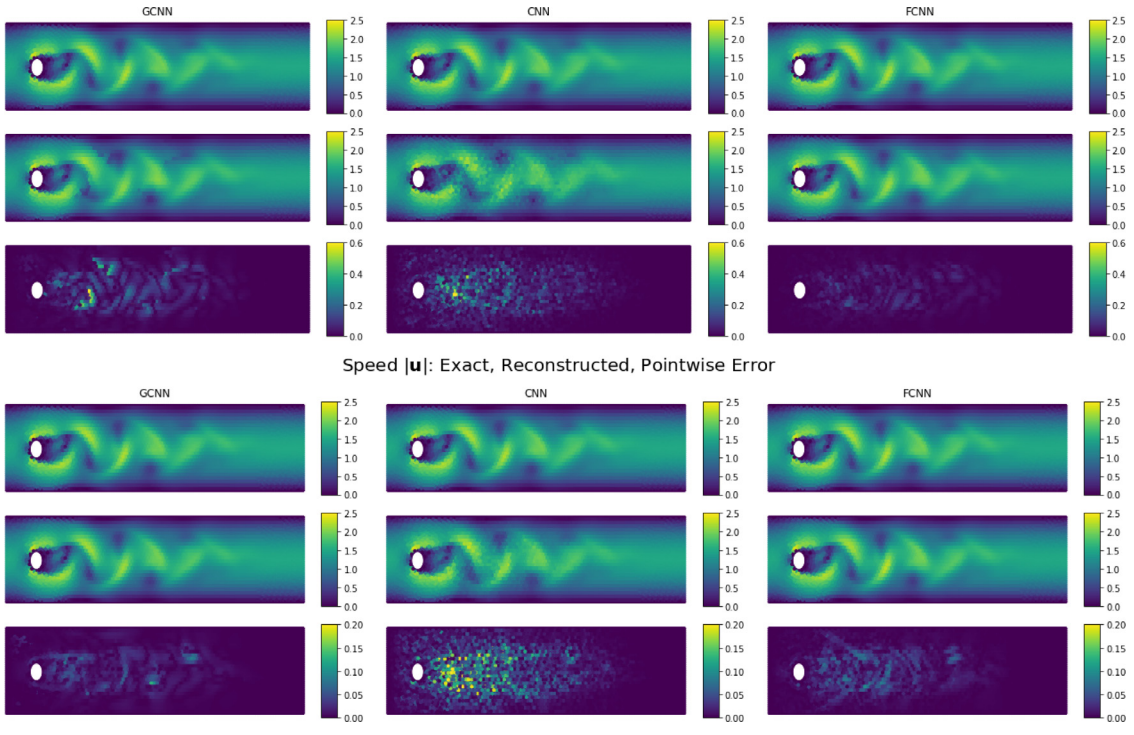

Fig. 13. Solutions to the prediction problem with $n = 32$ (top) and solutions to the compression problem with $n = 64$ (bottom) for the parameter $(t, \boldsymbol{\mu}) = \begin{pmatrix} 0.99 & 180 \end{pmatrix}^\top$.

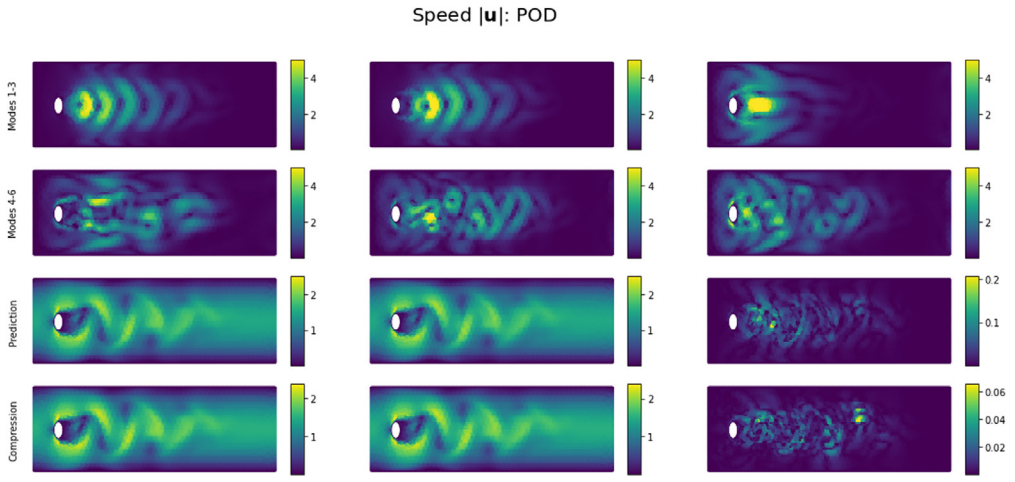

Fig. 14. Norm of the first six POD modes (rows 1 and 2) along with solutions to the prediction (row 3) and compression (row 4) problems when $(t, \boldsymbol{\mu}) = \begin{pmatrix} 0.99 & 180 \end{pmatrix}^\top$. Rows 3–4 contain the exact solution (left), the POD approximation (middle), and the pointwise error (right).

improving upon standard CNN-ROMs while maintaining a memory cost comparable to linear techniques such as POD. Moreover, it seems that the standard CAE is highly performing at low values of n but should be avoided when the model domain is irregular, while the fully connected autoencoder exhibits strong overall performance at a higher memory cost than the alternatives. It is also remarkable that traditional POD-based ROMs, while not strictly data-driven, can still be fast and effective when the latent dimension becomes high enough and typically exhibit more predictable dependence on dimension than the nonlinear network-based methods examined recently. Future work will examine how the proposed GCNN autoencoder can be combined with traditional PDE-based numerical solvers to generate solutions at unknown parameter configurations without the need for a fully connected prediction network, potentially eliminating the noted accuracy bottleneck arising from this part of the ROM.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Max Gunzburger reports financial support was provided by US Department of Energy. Lili Ju reports financial support was provided by US Department of Energy.

Acknowledgment

This work is partially supported by U.S. Department of Energy Scientific Discovery through Advanced Computing under grants DE-SC0020270 and DE-SC0020418.

Appendix. Galerkin POD ROMs

We briefly describe the Galerkin ROMs based on POD used for the examples. Note that Einstein summation is used throughout, so that any index which appears both up and down in a tensor expression is implicitly summed over its appropriate range.

A.1. 1-D Inviscid Burger's equation

To generate the POD-ROM based on the finite difference discretization in Section 4.2, the snapshot matrix $\mathbf{S} = \left(s_j^i\right)$ with entries $s_j^i = w^i(t_j, \boldsymbol{\mu}_j)$ is first preprocessed by subtracting the relevant initial condition from each column, generating $\bar{\mathbf{S}} = \left(\bar{s}_j^i\right)$ with $\bar{s}_j^i = s_j^i - w^i(t_0, \boldsymbol{\mu}_j)$. Factoring this through the singular value decomposition $\bar{\mathbf{S}} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$,

it follows that the optimal n-dimensional linear solution subspace (in the sense of variance maximization) is the span of the matrix $\mathbf{U}_n$ containing the first n columns of $\mathbf{U}$. The POD approximation $\tilde{\mathbf{w}} \approx \mathbf{w}$ is then expressed as

$$\tilde{\mathbf{w}} = \mathbf{w}_0 + \mathbf{U}_n \hat{\mathbf{w}},$$

where $\mathbf{w}_0(\boldsymbol{\mu}) = \mathbf{w}(t_0, \boldsymbol{\mu})$ and $\hat{\mathbf{w}} \in \mathbb{R}^n$ denotes the reduced-order state. Substituting this into the (discretization of) system (7) yields the reduced-order system

$$\begin{aligned} \frac{\hat{\mathbf{w}}_{k+1} - \hat{\mathbf{w}}_k}{\Delta t} + \frac{1}{2}\left(\mathbf{a} + \mathbf{B}\hat{\mathbf{w}}_k + \mathbf{C}(\hat{\mathbf{w}}_k, \hat{\mathbf{w}}_k)\right) &= 0.02\mathbf{U}_n^\top e^{\mu_2 \mathbf{x}}, \\ \hat{\mathbf{w}}_0 = \mathbf{U}_n^\top(\mathbf{w}_0 - \mathbf{w}_0) &= \mathbf{0}. \end{aligned} \tag{A.1}$$

where $0 \le k \le N_t - 1$, $e^{\mu_2 \mathbf{x}}$ acts component-wise, and $\mathbf{a}, \mathbf{B}, \mathbf{C}$ are vector-valued quantities which can be precomputed. In particular, let $\mathbf{D}_x \in \mathbb{R}^{N \times N}$ denote the bidiagonal forward difference matrix with entries $\{0, 1/\Delta x, \ldots, 1/\Delta x\}$ on its diagonal and $\{-1/\Delta x, \ldots, -1/\Delta x\}$ on its subdiagonal. A straightforward computation then yields

$$\begin{aligned} \mathbf{a} &= \mathbf{U}_n^\top \mathbf{D}_x (\mathbf{w}_0)^2, \\ \mathbf{B} &= 2\mathbf{U}_n^\top \mathbf{D}_x \mathrm{Diag}(\mathbf{w}_0)\mathbf{U}_n, \\ \mathbf{C} &= \mathbf{U}_n^\top \mathbf{D}_x \mathrm{diag}_{1,3}(\mathbf{U}_n \otimes \mathbf{U}_n), \end{aligned}$$

where $(\mathbf{w}_0)^2$ is a component-wise operation taking place in $\mathbb{R}^N$, $\mathrm{Diag}(\mathbf{w}_0)$ indicates construction of the diagonal matrix built from $\mathbf{w}_0$, and $\mathrm{diag}_{1,3}(\mathbf{U}_n \otimes \mathbf{U}_n)$ indicates extraction of the "diagonal" tensor formed from $\mathbf{U}_n \otimes \mathbf{U}_n$ when components 1 and 3 are the same, i.e. the rank 3 tensor with components $u^i_j u^i_k$. To generate the POD-ROM solution to the prediction problem at a particular $\boldsymbol{\mu}$ in the test set, the low-dimensional system (A.1) is solved for the reduced-order state $\hat{\mathbf{w}} \in \mathbb{R}^{n \times N_t}$, and the high-dimensional solution is rebuilt as $\tilde{\mathbf{w}} = \mathbf{w}_0 + \mathbf{U}_n \hat{\mathbf{w}}$ (where $\mathbf{w}_0$ is the relevant initial condition). Solutions to the compression problem are generated through simple projection/expansion as $\tilde{\mathbf{w}} = \mathbf{w}_0 + \mathbf{U}_n \mathbf{U}_n^\top (\mathbf{w} - \mathbf{w}_0)$.

A.2. Unsteady 2-D Navier–Stokes

The POD-ROM for the Navier–Stokes equations is based on the finite element discretization used to generate the FOM snapshot data. Beginning with the continuous velocity $\mathbf{u}$, the Galerkin assumption in this case is that

$$\mathbf{u}(x, t) \approx \tilde{\mathbf{u}}(x, t) = \bar{\mathbf{u}}(x) + u^I(t)\boldsymbol{\psi}_I(x)$$

for a time invariant "central velocity" $\bar{\mathbf{u}}$ and N shape functions $\boldsymbol{\psi}_I$ localized on the discrete domain. From this, POD seeks a collection of n variance-maximizing basis functions $\boldsymbol{\varphi}_a(x)$ such that $u^I(t) = \phi^I_a \hat{u}^a(t)$ and hence $u^I \boldsymbol{\psi}_I = \hat{u}^a \phi^I_a \boldsymbol{\psi}_I = \hat{u}^a \boldsymbol{\varphi}_a$. Discretely, it is useful to consider the velocity $\mathbf{u}$ as a $2N$-length vector of nodal values, where the components u_1, u_2 are vertically concatenated. In this case, the mean velocity $\bar{\mathbf{u}}$ is also a $2N$-vector containing the mean of each component separately, and the FOM snapshot matrix is a $2N \times N_s$ array.

Using $\mathbf{M} = \mathbf{R}^\top \mathbf{R}$ to denote the $(2N \times 2N)$ block diagonal mass matrix and its Cholesky factorization, it can be shown that the POD basis is related to the left singular vectors of the modified snapshot matrix $\tilde{\mathbf{S}} = \mathbf{R}\mathbf{S} = \tilde{\boldsymbol{\Phi}}\boldsymbol{\Sigma}\mathbf{V}^\top$ where $\mathbf{S} = (s^I_\alpha) \in \mathbb{R}^{2N \times N_s}$ is the mean-centered snapshot matrix with entries $s^I_\alpha = u^I(t_\alpha, \boldsymbol{\mu}_\alpha) - \bar{u}^I(\boldsymbol{\mu}_\alpha)$. In particular, the first n columns $\boldsymbol{\Phi}_n$ of the matrix $\boldsymbol{\Phi}$ satisfying $\mathbf{R}\boldsymbol{\Phi} = \tilde{\boldsymbol{\Phi}}$ are $\mathbf{M}$-orthonormal by construction and comprise the dimension n POD basis for $\mathbf{S}$. This represents the best possible n-dimensional linear approximation to snapshot dynamics, and solutions to the compression problem are readily constructed from it as

$$\tilde{\mathbf{u}} = \bar{\mathbf{u}} + \boldsymbol{\Phi}_n \boldsymbol{\Phi}_n^\top \mathbf{M}\,(\mathbf{u} - \bar{\mathbf{u}})\,,$$

where $\mathbf{u}, \bar{\mathbf{u}}$ can now depend on arbitrary $(t, \boldsymbol{\mu})$ pairs.

Using $(\cdot, \cdot)$ to denote the L^2 inner product on the domain Ω, integrating the Navier–Stokes equations by parts and using the boundary conditions yields the familiar weak form

$$\begin{aligned} (\mathbf{u}_t, \boldsymbol{\psi}) + \nu\,(\nabla \mathbf{u}, \nabla \boldsymbol{\psi}) + (\mathbf{u} \cdot \nabla \mathbf{u}, \boldsymbol{\psi}) - (p, \nabla \cdot \boldsymbol{\psi}) &= 0, \qquad \boldsymbol{\psi} \in H^1_0(\Omega; \mathbb{R}^2), \\ -(\eta, \nabla \cdot \mathbf{u}) &= 0, \qquad \eta \in L^2(\Omega), \end{aligned}$$

which represents the full-order system. A reduced-order system can be constructed in a similar fashion by using the POD decomposition. Substituting $\tilde{\mathbf{u}} \approx \mathbf{u}$ into the FOM, symmetrizing the weak Laplace term, and testing against the POD basis $\boldsymbol{\Phi}_n$ yields

$$(\tilde{\mathbf{u}}_t, \boldsymbol{\varphi}) + \nu \left(\nabla\tilde{\mathbf{u}} + \nabla\tilde{\mathbf{u}}^\top, \nabla\boldsymbol{\varphi}\right) + (\tilde{\mathbf{u}} \cdot \nabla\tilde{\mathbf{u}}, \boldsymbol{\varphi}) = 0, \qquad \boldsymbol{\varphi} \in \boldsymbol{\Phi}_n,$$

where $\nabla \cdot \nabla\mathbf{u}^\top = u^{ijk}\mathbf{e}_k \cdot (\mathbf{e}_i \otimes \mathbf{e}_j) = u^{ij}{}_i\mathbf{e}_j = \partial^j u^i_i \mathbf{e}_j = \nabla(\nabla \cdot \mathbf{u}) = 0$ since the mixed partial derivatives of $\mathbf{u}$ commute. Note that incompressibility of $\tilde{\mathbf{u}}$ is automatically satisfied as the mean velocity $\bar{\mathbf{u}}$ and POD basis $\boldsymbol{\varphi}$ are linear combinations of the snapshots. Moreover, in the ROM it is no longer necessary to keep track of the pressure p. Further computation using the $(\cdot,\cdot)$-orthogonality of the POD basis yields the reduced-order ODE system

$$\hat{u}_{c,t} = -r_c - \hat{u}^a S_{ac} - \hat{u}^b \hat{u}^a T_{bac},$$

for the components of $\hat{\mathbf{u}}$, where the quantities

$$\begin{aligned} r_c &= \left(\bar{\mathbf{u}} \cdot \nabla\bar{\mathbf{u}}, \boldsymbol{\varphi}_c\right) + \nu \left(\nabla\bar{\mathbf{u}} + \nabla\bar{\mathbf{u}}^\top, \nabla\boldsymbol{\varphi}_c\right), \\ S_{ac} &= \left(\bar{\mathbf{u}} \cdot \nabla\boldsymbol{\varphi}_a + \boldsymbol{\varphi}_a \cdot \nabla\bar{\mathbf{u}}, \boldsymbol{\varphi}_c\right) + \nu \left(\nabla\boldsymbol{\varphi}_a + \nabla\boldsymbol{\varphi}_a^\top, \nabla\boldsymbol{\varphi}_c\right), \\ T_{bac} &= \left(\boldsymbol{\varphi}_b \cdot \nabla\boldsymbol{\varphi}_a, \boldsymbol{\varphi}_c\right), \end{aligned}$$

are precomputed. To generate the POD comparisons in Section 4.4, the above system is solved using a simple forward Euler scheme with $\Delta t = 2.5 \times 10^{-5}$, which is 100 times smaller than the time step between FOM snapshots. Solutions to the prediction problem are then given simply by $\tilde{\mathbf{u}} = \bar{\mathbf{u}} + \boldsymbol{\Phi}_n\hat{\mathbf{u}}$. Note that in this case the notion of error is best captured by the norm $\mathbf{u} \mapsto \left(\mathbf{u}^\top\mathbf{M}\mathbf{u}\right)^{1/2}$ defined by the mass matrix, so all relative error metrics are reported in this norm.

References

[1] M. Milano, P. Koumoutsakos, Neural network modeling for near wall turbulent flow, J. Comput. Phys. 182 (1) (2002) 1–26.

[2] K. Kashima, Nonlinear model reduction by deep autoencoder of noise response data, in: 2016 IEEE 55th Conference on Decision and Control (CDC), IEEE, 2016, pp. 5750–5755.

[3] D. Hartman, L.K. Mestha, A deep learning framework for model reduction of dynamical systems, in: 2017 IEEE Conference on Control Technology and Applications (CCTA), IEEE, 2017, pp. 1917–1922.

[4] K. Lee, K.T. Carlberg, Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders, J. Comput. Phys. 404 (2020) 108973.

[5] K. Fukami, K. Hasegawa, T. Nakamura, M. Morimoto, K. Fukagata, Model order reduction with neural networks: Application to laminar and turbulent flows, 2020, arXiv preprint arXiv:2011.10277.

[6] H. Eivazi, H. Veisi, M.H. Naderi, V. Esfahanian, Deep neural networks for nonlinear model order reduction of unsteady flows, Phys. Fluids 32 (10) (2020) 105104.

[7] S. Fresca, A. Manzoni, L. Dedé, A comprehensive deep learning-based approach to reduced order modeling of nonlinear time-dependent parametrized PDEs, J. Sci. Comput. 87 (2) (2021) 1–36.

[8] R. Maulik, B. Lusch, P. Balaprakash, Reduced-order modeling of advection-dominated systems with recurrent neural networks and convolutional autoencoders, Phys. Fluids 33 (3) (2021) 037106.

[9] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, S.Y. Philip, A comprehensive survey on graph neural networks, IEEE Trans. Neural Netw. Learn. Syst. 32 (1) (2020) 4–24.

[10] S. Ioffe, C. Szegedy, Batch normalization: Accelerating deep network training by reducing internal covariate shift, in: International Conference on Machine Learning, PMLR, 2015, pp. 448–456.

[11] M. Chen, Z. Wei, Z. Huang, B. Ding, Y. Li, Simple and deep graph convolutional networks, in: International Conference on Machine Learning, PMLR, 2020, pp. 1725–1735.

[12] V. Dumoulin, F. Visin, A guide to convolution arithmetic for deep learning, 2016, arXiv preprint arXiv:1603.07285.

[13] H. Fu, H. Wang, Z. Wang, POD/DEIM reduced-order modeling of time-fractional partial differential equations with applications in parameter identification, J. Sci. Comput. 74 (1) (2018) 220–243.

[14] B. Bollobás, Modern Graph Theory, Vol. 184, Springer Science & Business Media, 2013.

[15] M. Defferrard, X. Bresson, P. Vandergheynst, Convolutional neural networks on graphs with fast localized spectral filtering, Adv. Neural Inf. Process. Syst. 29 (2016) 3844–3852.

[16] D.K. Hammond, P. Vandergheynst, R. Gribonval, Wavelets on graphs via spectral graph theory, Appl. Comput. Harmon. Anal. 30 (2) (2011) 129–150.

[17] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, 2016, arXiv preprint arXiv:1609.02907.

[18] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, K. Weinberger, Simplifying graph convolutional networks, in: International Conference on Machine Learning, PMLR, 2019, pp. 6861–6871.

[19] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.

[20] X. Bresson, T. Laurent, Residual gated graph convnets, 2017, arXiv preprint arXiv:1711.07553.
[21] J. Gilmer, S.S. Schoenholz, P.F. Riley, O. Vinyals, G.E. Dahl, Neural message passing for quantum chemistry, in: International Conference on Machine Learning, PMLR, 2017, pp. 1263–1272.
[22] C.R. Qi, L. Yi, H. Su, L.J. Guibas, Pointnet++: Deep hierarchical feature learning on point sets in a metric space, 2017, arXiv preprint arXiv:1706.02413.
[23] Z. Ma, M. Li, Y. Wang, PAN: path integral based convolution for deep graph neural networks, 2019, arXiv preprint arXiv:1904.10996.
[24] Y. Wang, Y. Sun, Z. Liu, S.E. Sarma, M.M. Bronstein, J.M. Solomon, Dynamic graph cnn for learning on point clouds, ACM Trans. Graph. 38 (5) (2019) 1–12.
[25] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: An imperative style, high-performance deep learning library, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, R. Garnett (Eds.), Advances in Neural Information Processing Systems 32, Curran Associates, Inc., 2019, pp. 8024–8035.
[26] M. Fey, J.E. Lenssen, Fast graph representation learning with pyTorch geometric, 2019, arXiv:1903.02428.
[27] H.C. Elman, A. Ramage, D.J. Silvester, Algorithm 866: IFISS, a matlab toolbox for modelling incompressible flow, ACM Trans. Math. Softw. 33 (2) (2007) 14–es.
[28] M. Schäfer, S. Turek, F. Durst, E. Krause, R. Rannacher, Benchmark computations of laminar flow around a cylinder, in: Flow Simulation with High-Performance Computers II, Springer, 1996, pp. 547–566.